# Lost and Found

***Release 1.0.0***

**Mary Poppins**

**Mar 12, 2026**

# CONTENTS

CHAPTER

ONE

# CORE

The core is built around an immutable state object that includes a replica of the items table, as well as any transient application specific state like search box contents. Any update to the state creates a new state object with the applied changes, other areas of the application can observe these changes e.g. models and the database layer.

This state is broken down into individual entities, like items, and search parameters. These entities can then be composed into a hierarchy which handles processing updates to individual entities. The UI and the database can subscribe to updates to a specific entity, as well as a specific property on a certain entity.

CHAPTER

TWO

# UI

Models monitor changes to the state object and shrink them down to individual observables, which can be connected to the UI using view models. These do not contain any UI code and are be unit tested. View models handle connecting observables from the staging area to UI, for example connecting the name of an item to an element in the treeview. They also construct the UI by composing other view models i.e. ButtonViewModel. Views contain the actual UI logic i.e. tkinter code. These only exist for primitive widgets and get composed in view models to create more complex UI. This minimises the amount of tkinter code.

CHAPTER

THREE

# DATABASE

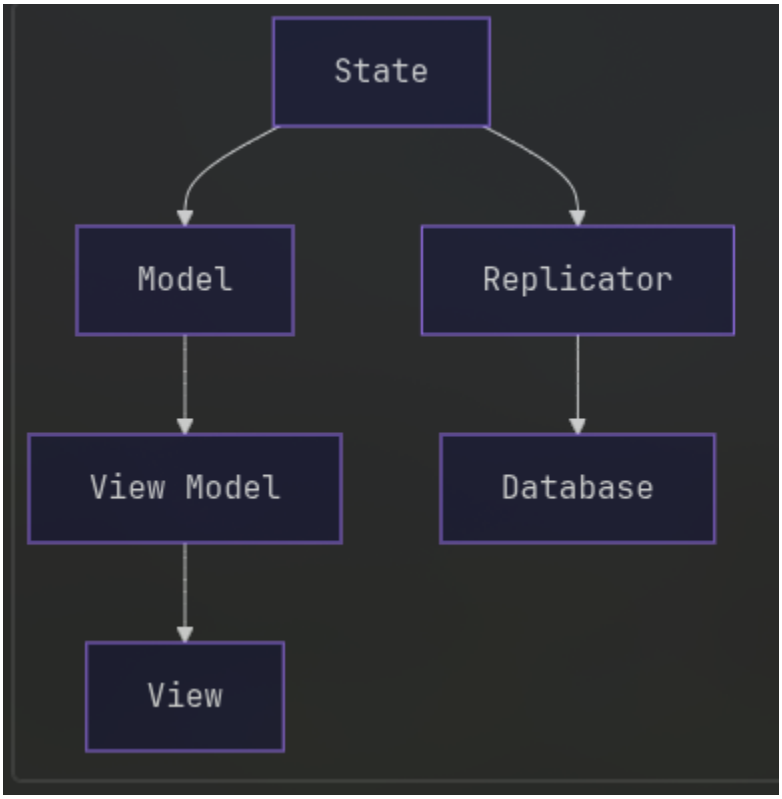
The database is injected as a service, and has multiple implementations e.g. file, or in memory. A database can execute SQL queries. Replicators handle executing queries to update the database when application state is modified. Replicators also initialise the application state based on the contents of the database.

Custom types can be stored into the database using converters, which tell the database how to convert the type to and from a bytes object. Converters can be registered into the database at startup to handle custom types like datetimes, or item category enums.

CHAPTER

# FOUR

# ARCHITECTURE DIAGRAM




*lost_and_found*

## 4.1 lost_and_found

### Functions

*main*()

### 4.1.1 lost_and_found.main

lost_and_found.**main**() → None

#### Modules

| |
|---|
| *core* |
| *db* |
| *state* |
| *ui* |

### 4.1.2 lost_and_found.core

#### Modules

| |
|---|
| *immutable* |
| *observable* |

#### lost_and_found.core.immutable

#### Classes

| | |
|---|---|
| *ImmutableList*([value]) | An immutable, tuple-backed list-like container. |

#### lost_and_found.core.immutable.ImmutableList

**class** lost_and_found.core.immutable.**ImmutableList**(*value: tuple[T, ...] = ()*)

Bases: Generic

An immutable, tuple-backed list-like container.

*ImmutableList* stores its items in a tuple and returns new instances on mutations such as *append* and *remove*.

**__init__**(*value: tuple[T, ...] = ()*) → None

##### Methods

| | |
|---|---|
| *__init__*([value]) | |
| *append*(value) | Return a new *ImmutableList* with *value* appended. |
| *remove*(value) | Return a new *ImmutableList* with all occurrences of *value* removed. |

##### Attributes

| |
|---|
| value |

**append**(*value: T*) → *ImmutableList*[T]

Return a new *ImmutableList* with *value* appended.

The original instance is not modified.

**remove**(*value: T*) → *ImmutableList*[T]

Return a new *ImmutableList* with all occurrences of *value* removed.

### lost_and_found.core.observable

### Classes

| | |
|---|---|
| *Observable*() | Base observable interface. |
| *ObservableFilter*(parent, predicate) | An observable that forwards values from a parent only when a predicate passes. |
| *Property*(initial) | A simple mutable *ValueObservable*. |
| *Trigger*() | An observable that only notifies subscribers when *trigger* is called. |
| *ValueObservable*() | An *Observable* that has a current *value*. |

### lost_and_found.core.observable.Observable

**class** lost_and_found.core.observable.**Observable**

Bases: ABC, Generic

Base observable interface.

Subclasses should implement *subscribe* to accept a callable that will be invoked whenever the observable emits a value.

**__init__**()

#### Methods

| | |
|---|---|
| *__init__*() | |
| *filter*(predicate) | Return an observable that only emits values for which *predicate* is true. |
| *start_with*(initial) | Create a *ValueObservable* that starts with *initial* and follows this observable's updates. |
| *subscribe*(subscriber) | Register *subscriber* to receive updates from this observable. |

**filter**(*predicate: Callable[[T], bool]*) → *Observable*[T]

Return an observable that only emits values for which *predicate* is true.

**start_with**(*initial: T*) → *ValueObservable*[T]

Create a *ValueObservable* that starts with *initial* and follows this observable's updates.

**abstractmethod subscribe**(*subscriber: Callable[[T], None]*) → None

Register *subscriber* to receive updates from this observable.

### lost_and_found.core.observable.ObservableFilter

**class** lost_and_found.core.observable.**ObservableFilter**(*parent: Observable[T], predicate: Callable[[T], bool]*)

Bases: *Observable*, Generic

An observable that forwards values from a parent only when a predicate passes.

**__init__**(*parent: Observable[T], predicate: Callable[[T], bool]*)

#### Methods

| | |
|---|---|
| *__init__*(parent, predicate) | |
| *filter*(predicate) | Return an observable that only emits values for which *predicate* is true. |
| *start_with*(initial) | Create a *ValueObservable* that starts with *initial* and follows this observable's updates. |
| *subscribe*(subscriber) | Register a subscriber to receive filtered values. |

**filter**(*predicate: Callable[[T], bool]*) → *Observable*[T]

Return an observable that only emits values for which *predicate* is true.

**start_with**(*initial: T*) → *ValueObservable*[T]

Create a *ValueObservable* that starts with *initial* and follows this observable's updates.

**subscribe**(*subscriber: Callable[[T], None]*) → None

Register a subscriber to receive filtered values.

### lost_and_found.core.observable.Property

**class** lost_and_found.core.observable.**Property**(*initial: T*)

Bases: *ValueObservable*, Generic

A simple mutable *ValueObservable*.

Calling *update* changes the stored value and notifies subscribers. When a subscriber registers it is immediately called with the current value.

**__init__**(*initial: T*)

#### Methods

| | |
|---|---|
| *__init__*(initial) | |
| *combine*(a, b) | Combine two `ValueObservable`s into one that yields pairs of values. |
| *filter*(predicate) | Return an observable that only emits values for which *predicate* is true. |
| *map*(transform) | Return a mapped *ValueObservable* whose value is *transform(value)*, and which updates whenever the source changes. |
| *on_change_only*() | Return a *ValueObservable* that filters out consecutive duplicate values, emitting only on actual changes. |

continues on next page

Table 11 – continued from previous page

| | |
|---|---|
| *start_with*(initial) | Create a *ValueObservable* that starts with *initial* and follows this observable's updates. |
| *subscribe*(subscriber) | Subscribe and immediately notify with the current *value*. |
| *update*(new_value) | Set *new_value* and notify subscribers. |

### Attributes

| | |
|---|---|
| *value* | The current stored value. |

**static combine**(*a:* ValueObservable, *b:* ValueObservable) → *ValueObservable*[tuple[TA, TB]]

Combine two `ValueObservable`s into one that yields pairs of values.

The returned *ValueObservable* updates whenever either source updates.

**filter**(*predicate: Callable[[T], bool]*) → *Observable*[T]

Return an observable that only emits values for which *predicate* is true.

**map**(*transform: Callable[[T], TResult]*) → *ValueObservable*[TResult]

Return a mapped *ValueObservable* whose value is *transform(value)*, and which updates whenever the source changes.

**on_change_only**() → *ValueObservable*[T]

Return a *ValueObservable* that filters out consecutive duplicate values, emitting only on actual changes.

**start_with**(*initial: T*) → *ValueObservable*[T]

Create a *ValueObservable* that starts with *initial* and follows this observable's updates.

**subscribe**(*subscriber: Callable[[T], None]*) → None

Subscribe and immediately notify with the current *value*.

**update**(*new_value: T*) → None

Set *new_value* and notify subscribers.

**property value: T**

The current stored value.

## lost_and_found.core.observable.Trigger

**class** lost_and_found.core.observable.**Trigger**

Bases: *Observable*, Generic

An observable that only notifies subscribers when *trigger* is called.

Useful for one-off events rather than maintaining a current value.

**__init__**() → None

### Methods

| | |
|---|---|
| *__init__*() | |

continues on next page

Table 13 – continued from previous page

| | |
|---|---|
| *filter*(predicate) | Return an observable that only emits values for which *predicate* is true. |
| *start_with*(initial) | Create a *ValueObservable* that starts with *initial* and follows this observable's updates. |
| *subscribe*(subscriber) | Register a subscriber to be invoked when *trigger* is called. |
| *trigger*(value) | Invoke all subscribers with *value*. |

**filter**(*predicate: Callable[[T], bool]*) → *Observable*[T]

Return an observable that only emits values for which *predicate* is true.

**start_with**(*initial: T*) → *ValueObservable*[T]

Create a *ValueObservable* that starts with *initial* and follows this observable's updates.

**subscribe**(*subscriber: Callable[[T], None]*) → None

Register a subscriber to be invoked when *trigger* is called.

**trigger**(*value: T*) → None

Invoke all subscribers with *value*.

### lost_and_found.core.observable.ValueObservable

**class** lost_and_found.core.observable.**ValueObservable**

Bases: *Observable*, Generic

An *Observable* that has a current *value*.

*ValueObservable* implementations expose a *value* property and provide convenience helpers like *map* and *combine*.

**__init__**()

#### Methods

| | |
|---|---|
| *__init__*() | |
| *combine*(a, b) | Combine two `ValueObservable`s into one that yields pairs of values. |
| *filter*(predicate) | Return an observable that only emits values for which *predicate* is true. |
| *map*(transform) | Return a mapped *ValueObservable* whose value is *transform(value)*, and which updates whenever the source changes. |
| *on_change_only*() | Return a *ValueObservable* that filters out consecutive duplicate values, emitting only on actual changes. |
| *start_with*(initial) | Create a *ValueObservable* that starts with *initial* and follows this observable's updates. |
| *subscribe*(subscriber) | Register *subscriber* to receive updates from this observable. |

#### Attributes

| | |
|---|---|
| *value* | The current value of the observable. |

**static combine**(*a:* ValueObservable, *b:* ValueObservable) → *ValueObservable*[tuple[TA, TB]]

Combine two `ValueObservable`s into one that yields pairs of values.

The returned *ValueObservable* updates whenever either source updates.

**filter**(*predicate: Callable[[T], bool]*) → *Observable*[T]

Return an observable that only emits values for which *predicate* is true.

**map**(*transform: Callable[[T], TResult]*) → *ValueObservable*[TResult]

Return a mapped *ValueObservable* whose value is *transform(value)*, and which updates whenever the source changes.

**on_change_only**() → *ValueObservable*[T]

Return a *ValueObservable* that filters out consecutive duplicate values, emitting only on actual changes.

**start_with**(*initial: T*) → *ValueObservable*[T]

Create a *ValueObservable* that starts with *initial* and follows this observable's updates.

**abstractmethod subscribe**(*subscriber: Callable[[T], None]*) → None

Register *subscriber* to receive updates from this observable.

**abstract property value: T**

The current value of the observable.

### 4.1.3 lost_and_found.db

#### Modules

| |
|---|
| *converters* |
| *database* |
| *replicator* |
| *tables* |

#### lost_and_found.db.converters

#### Classes

| | |
|---|---|
| *CategoryConverter*() | Converter for the *Category* enum value used by *Item*. |
| *Converter*() | Base class for converting values to/from bytes/strings for DB. |
| *DatetimeConverter*() | Converter for *datetime* objects using ISO format. |

#### lost_and_found.db.converters.CategoryConverter

class lost_and_found.db.converters.**CategoryConverter**

Bases: *Converter*[*Category*]

Converter for the *Category* enum value used by *Item*.

**__init__**()

#### Methods

| |
|---|
| *__init__*() |
| from_bytes(bytes) |
| to_str(value) |
| type() |

### lost_and_found.db.converters.Converter

class lost_and_found.db.converters.**Converter**

Bases: ABC, Generic

Base class for converting values to/from bytes/strings for DB.

Concrete implementations must provide the Python *type*, a *to_str* adapter and a *from_bytes* parser used when registering with *sqlite3*.

**__init__**()

#### Methods

| |
|---|
| *__init__*() |
| from_bytes(bytes) |
| to_str(value) |
| type() |

### lost_and_found.db.converters.DatetimeConverter

class lost_and_found.db.converters.**DatetimeConverter**

Bases: *Converter*[datetime]

Converter for *datetime* objects using ISO format.

**__init__**()

#### Methods

| |
|---|
| *__init__*() |
| from_bytes(bytes) |
| to_str(value) |
| type() |

### lost_and_found.db.database

### Classes

| | |
|---|---|
| *Database*() | A base class interface for database implementations. |
| *FileDatabase*(path) | A *SqliteDatabase* backed by a file on disk. |
| *InMemoryDatabase*() | An in-memory *SqliteDatabase* useful for tests and ephemeral use. |
| *SqliteDatabase*(conn) | SQLite-backed *Database* implementation. |

### lost_and_found.db.database.Database

**class** lost_and_found.db.database.**Database**

Bases: ABC

A base class interface for database implementations.

**__init__**()

#### Methods

| | |
|---|---|
| *__init__*() | |
| *add_table*(table) | Create and register a *Table* instance. |
| *register_converter*(converter) | Register a converter type so custom types can be stored. |
| *replicate_table_to*(table, model) | Start replicating changes from *model* into *table*. |

**abstractmethod add_table**(*table: Type[Table]*) → None

Create and register a *Table* instance.

**abstractmethod register_converter**(*converter: Type[Converter]*) → None

Register a converter type so custom types can be stored.

**abstractmethod replicate_table_to**(*table: Type[Table]*, *model:* ListModel) → None

Start replicating changes from *model* into *table*.

### lost_and_found.db.database.FileDatabase

**class** lost_and_found.db.database.**FileDatabase**(*path: str*)

Bases: *SqliteDatabase*

A *SqliteDatabase* backed by a file on disk.

**__init__**(*path: str*) → None

#### Methods

| | |
|---|---|
| *__init__*(path) | |
| *add_table*(table) | Instantiate and create the database table for *table*. |
| *register_converter*(converter) | Register adapter and converter functions with *sqlite3*. |
| *replicate_table_to*(table, model) | Attach a *Replicator* to mirror a model into a DB table. |

**add_table**(*table: Type[Table]*) → None

Instantiate and create the database table for *table*.

**register_converter**(*converter: Type[Converter]*) → None

Register adapter and converter functions with *sqlite3*.

**replicate_table_to**(*table: Type[Table]*, *model:* ListModel) → None

Attach a *Replicator* to mirror a model into a DB table.

### lost_and_found.db.database.InMemoryDatabase

**class** lost_and_found.db.database.**InMemoryDatabase**

Bases: *SqliteDatabase*

An in-memory *SqliteDatabase* useful for tests and ephemeral use.

**__init__**() → None

#### Methods

| | |
|---|---|
| *__init__*() | |
| *add_table*(table) | Instantiate and create the database table for *table*. |
| *register_converter*(converter) | Register adapter and converter functions with *sqlite3*. |
| *replicate_table_to*(table, model) | Attach a *Replicator* to mirror a model into a DB table. |

**add_table**(*table: Type[Table]*) → None

Instantiate and create the database table for *table*.

**register_converter**(*converter: Type[Converter]*) → None

Register adapter and converter functions with *sqlite3*.

**replicate_table_to**(*table: Type[Table]*, *model:* ListModel) → None

Attach a *Replicator* to mirror a model into a DB table.

### lost_and_found.db.database.SqliteDatabase

**class** lost_and_found.db.database.**SqliteDatabase**(*conn: Connection*)

Bases: *Database*

SQLite-backed *Database* implementation.

This class wraps a sqlite3 connection and exposes simple helpers to register converters and to create table instances. Tables created via *add_table* will have their schema created immediately.

**__init__**(*conn: Connection*) → None

#### Methods

| | |
|---|---|
| *__init__*(conn) | |
| *add_table*(table) | Instantiate and create the database table for *table*. |
| *register_converter*(converter) | Register adapter and converter functions with *sqlite3*. |
| *replicate_table_to*(table, model) | Attach a *Replicator* to mirror a model into a DB table. |

**add_table**(*table: Type[Table]*) → None

Instantiate and create the database table for *table*.

**register_converter**(*converter: Type[Converter]*) → None

Register adapter and converter functions with *sqlite3*.

**replicate_table_to**(*table: Type[Table]*, *model:* ListModel) → None

Attach a *Replicator* to mirror a model into a DB table.

### lost_and_found.db.replicator

### Classes

| | |
|---|---|
| *Replicator*(model, table) | Listen to a *ListModel* and apply changes to a *Table*. |

### lost_and_found.db.replicator.Replicator

**class** lost_and_found.db.replicator.**Replicator**(*model:* ListModel, *table:* Table)

Bases: ABC, Generic

Listen to a *ListModel* and apply changes to a *Table*.

On construction the replicator seeds the model with existing rows from *table.select_all()* and subscribes to subsequent changes to replicate diffs into the table.

**__init__**(*model:* ListModel, *table:* Table) → None

#### Methods

| | |
|---|---|
| *__init__*(model, table) | |
| *diff*(previous, current) | Return three lists: (inserted, updated, deleted) between two states. |
| *on_change*(values) | Handle updated model values by computing and applying a diff. |

**static diff**(*previous:* ImmutableList, *current:* ImmutableList) → tuple[*ImmutableList*, *ImmutableList*, *ImmutableList*]

Return three lists: (inserted, updated, deleted) between two states.

Entities are compared by *id*. When an id is present in both but the object identity differs the entity is considered updated.

**on_change**(*values:* ImmutableList) → None

Handle updated model values by computing and applying a diff.

### lost_and_found.db.tables

### Classes

| | |
|---|---|
| *ItemsTable*(cursor) | Concrete table implementation for *Item* entities. |
| *Table*(cursor) | Base class representing a DB table for entities of type *T*. |

### lost_and_found.db.tables.ItemsTable

**class** lost_and_found.db.tables.**ItemsTable**(*cursor: Cursor*)

Bases: *Table*[*Item*]

Concrete table implementation for *Item* entities.

**__init__**(*cursor: Cursor*) → None

#### Methods

| | |
|---|---|
| *__init__*(cursor) | |
| *create_table*() | Execute the table creation SQL returned by *create_table_query*. |
| create_table_query() | |
| *delete*(value) | Remove the row that corresponds to *value*. |
| delete_query() | |
| from_row(id, row) | |
| *id*(pk) | Return the *EntityId* for a given primary key, creating one if none exists yet. |
| *insert*(value) | Insert *value* as a new row and record its primary key mapping. |
| insert_query() | |
| *pk*(id) | Return the primary key for an *EntityId* if known, otherwise *None*. |
| *select_all*() | Select all rows from the table and return them as an *ImmutableList* of entities. |
| select_all_query() | |
| to_row(value) | |
| *update*(value) | Update an existing row corresponding to *value*. |
| update_query() | |

**create_table**() → None

Execute the table creation SQL returned by *create_table_query*.

**delete**(*value: T*) → None

Remove the row that corresponds to *value*.

**id**(*pk: int*) → *EntityId*

Return the *EntityId* for a given primary key, creating one if none exists yet.

**insert**(*value: T*) → None

Insert *value* as a new row and record its primary key mapping.

**pk**(*id:* EntityId) → int | None

Return the primary key for an *EntityId* if known, otherwise *None*.

**select_all**() → *ImmutableList*

Select all rows from the table and return them as an *ImmutableList* of entities.

**update**(*value: T*) → None

Update an existing row corresponding to *value*.

### lost_and_found.db.tables.Table

**class** lost_and_found.db.tables.**Table**(*cursor: Cursor*)

Bases: ABC, Generic

Base class representing a DB table for entities of type *T*.

The class stores a cursor and maintains a mapping between SQLite primary keys and in-memory `EntityId`s so rows can be correlated with model entities.

**__init__**(*cursor: Cursor*) → None

#### Methods

| | |
|---|---|
| *__init__*(cursor) | |
| *create_table*() | Execute the table creation SQL returned by *create_table_query*. |
| create_table_query() | |
| *delete*(value) | Remove the row that corresponds to *value*. |
| delete_query() | |
| from_row(id, row) | |
| *id*(pk) | Return the *EntityId* for a given primary key, creating one if none exists yet. |
| *insert*(value) | Insert *value* as a new row and record its primary key mapping. |
| insert_query() | |
| *pk*(id) | Return the primary key for an *EntityId* if known, otherwise *None*. |
| *select_all*() | Select all rows from the table and return them as an *ImmutableList* of entities. |
| select_all_query() | |
| to_row(value) | |
| *update*(value) | Update an existing row corresponding to *value*. |
| update_query() | |

**create_table**() → None

Execute the table creation SQL returned by *create_table_query*.

**delete**(*value: T*) → None

Remove the row that corresponds to *value*.

**id**(*pk: int*) → *EntityId*

Return the *EntityId* for a given primary key, creating one if none exists yet.

**insert**(*value: T*) → None

Insert *value* as a new row and record its primary key mapping.

**pk**(*id:* EntityId) → int | None

Return the primary key for an *EntityId* if known, otherwise *None*.

**select_all**() → *ImmutableList*

Select all rows from the table and return them as an *ImmutableList* of entities.

**update**(*value: T*) → None

Update an existing row corresponding to *value*.

### 4.1.4 lost_and_found.state

#### Modules

| |
|---|
| *app* |
| *entity* |
| *item* |
| *model* |
| *search* |

#### lost_and_found.state.app

#### Classes

| | |
|---|---|
| *App*(id, children, items_id, search_params_id) | Root *Entity* for the application state that contains key children. |

#### lost_and_found.state.app.App

**class** lost_and_found.state.app.**App**(*id:* EntityId, *children:* ImmutableList*[*Entity*]*, *items_id:* EntityId, *search_params_id:* EntityId)

Bases: *Entity*

Root *Entity* for the application state that contains key children.

**__init__**(*id:* EntityId, *children:* ImmutableList*[*Entity*]*, *items_id:* EntityId, *search_params_id:* EntityId) → None

##### Methods

| | |
|---|---|
| *__init__*(id, children, items_id, ...) | |
| *get*(id) | Recursively find an entity with *id* in this subtree, or *None*. |
| new() | |
| *update_child*(child) | Return a new *App* with *child* replaced among its children. |

##### Attributes

| | |
|---|---|
| *items* | Return the *ListEntity* that stores *Item* instances. |
| *search_params* | Return the *SearchParams* entity used for filtering items. |
| items_id | |
| search_params_id | |
| id | |
| children | |

**get**(*id:* EntityId) → *Entity* | None

Recursively find an entity with *id* in this subtree, or *None*.

**property items:** *ListEntity*[*Item*]

Return the *ListEntity* that stores *Item* instances.

**property search_params:** *SearchParams*

Return the *SearchParams* entity used for filtering items.

**update_child**(*child:* Entity) → *App*

Return a new *App* with *child* replaced among its children.

### lost_and_found.state.entity

### Classes

| | |
|---|---|
| *Entity*(id, children) | Base class for state entities. |
| *EntityId*(id) | Lightweight wrapper around a UUID used to identify entities. |
| *Hierarchy*(root) | Wrap an entity tree and provide atomic update operations. |
| *ListEntity*(id, children[, items]) | An *Entity* that holds an ordered list of child entities in *items*. |

### lost_and_found.state.entity.Entity

**class** lost_and_found.state.entity.**Entity**(*id:* EntityId, *children:* ImmutableList*[*Entity*]*)

Bases: ABC

Base class for state entities.

Entities are immutable dataclasses containing an *id* and zero or more children. Subclasses must implement *update_child* to return a new instance with a replaced child entity.

**__init__**(*id:* EntityId, *children:* ImmutableList*[*Entity*]*) → None

#### Methods

| | |
|---|---|
| *__init__*(id, children) | |
| *get*(id) | Recursively find an entity with *id* in this subtree, or *None*. |
| update_child(child) | |

#### Attributes

| |
|---|
| id |
| children |

**get**(*id:* EntityId) → *Entity* | None

Recursively find an entity with *id* in this subtree, or *None*.

### lost_and_found.state.entity.EntityId

class lost_and_found.state.entity.**EntityId**(*id: UUID*)

Bases: object

Lightweight wrapper around a UUID used to identify entities.

**__init__**(*id: UUID*) → None

#### Methods

| | |
|---|---|
| *__init__*(id) | |
| *new*() | Create a new *EntityId* with a random UUID. |

#### Attributes

| |
|---|
| id |

**static new**() → *EntityId*

Create a new *EntityId* with a random UUID.

### lost_and_found.state.entity.Hierarchy

class lost_and_found.state.entity.**Hierarchy**(*root: T*)

Bases: Generic

Wrap an entity tree and provide atomic update operations.

**__init__**(*root: T*) → None

#### Methods

| | |
|---|---|
| *__init__*(root) | |
| *update*(entity) | Apply an update to the tree by replacing the matching entity. |

#### Attributes

| |
|---|
| observable |
| root |

**update**(*entity:* Entity) → None

Apply an update to the tree by replacing the matching entity.

### lost_and_found.state.entity.ListEntity

class lost_and_found.state.entity.**ListEntity**(*id:* EntityId, *children:* ImmutableList*[*Entity*]*, *items:* ImmutableList*[T]* = *ImmutableList(value=())*)

Bases: *Entity*, Generic

An *Entity* that holds an ordered list of child entities in *items*.

**__init__**(*id:* EntityId, *children:* ImmutableList*[*Entity*]*, *items:* ImmutableList*[T]* = *ImmutableList(value=())*) → None

### Methods

| | |
|---|---|
| *__init__*(id, children[, items]) | |
| *append*(entity) | Return a new *ListEntity* with *entity* appended to *items*. |
| *get*(id) | Recursively find an entity with *id* in this subtree, or *None*. |
| new() | |
| *remove_all*(to_remove) | Return a new *ListEntity* with *to_remove* removed. |
| *set*(items) | Replace the *items* collection and return a new *ListEntity*. |
| update_child(child) | |

### Attributes

| |
|---|
| items |
| id |
| children |

**append**(*entity: T*) → *ListEntity*[T]

Return a new *ListEntity* with *entity* appended to *items*.

**get**(*id:* EntityId) → *Entity* | None

Recursively find an entity with *id* in this subtree, or *None*.

**remove_all**(*to_remove:* ImmutableList*[T]*) → *ListEntity*[T]

Return a new *ListEntity* with *to_remove* removed.

**set**(*items:* ImmutableList*[T]*) → *ListEntity*[T]

Replace the *items* collection and return a new *ListEntity*.

## lost_and_found.state.item

### Classes

| | |
|---|---|
| *Category*(*values) | |
| *Item*(id, children, name, category, lost, ...) | Immutable representation of an item in the system. |

## lost_and_found.state.item.Category

**class** lost_and_found.state.item.**Category**(**values*)

Bases: Enum

**__init__**(**args*, ***kwds*)

### Attributes

| |
|---|
| ELECTRONICS |
| BOOKS |
| CLOTHING |

## lost_and_found.state.item.Item

**class** lost_and_found.state.item.**Item**(*id: EntityId*, *children: ImmutableList[Entity]*, *name: str*, *category: Category*, *lost: datetime*, *found: datetime | None*, *claimed: datetime | None*, *location: str | None*, *finder_email: str | None*, *owner_email: str | None*)

Bases: *Entity*

Immutable representation of an item in the system.

**__init__**(*id: EntityId*, *children: ImmutableList[Entity]*, *name: str*, *category: Category*, *lost: datetime*, *found: datetime | None*, *claimed: datetime | None*, *location: str | None*, *finder_email: str | None*, *owner_email: str | None*) → None

### Methods

| | |
|---|---|
| *__init__*(id, children, name, category, lost, ...) | |
| *claim*(claimed, owner_email) | Claim a found item. |
| *create_found_item*(name, category, found, ...) | Create a new found item. |
| *create_lost_item*(name, category, lost, ...) | Create a new lost item. |
| *get*(id) | Recursively find an entity with *id* in this subtree, or *None*. |
| *mark_as_found*(found, location, finder_email) | Mark an item as found. |
| update_child(child) | |

### Attributes

| |
|---|
| name |
| category |
| lost |
| found |
| claimed |
| location |
| finder_email |
| owner_email |
| id |
| children |

**claim**(*claimed: datetime*, *owner_email: str*) → *Item*

Claim a found item. Item must have been found and not claimed already.

**Parameters**

- **self** – The item to claim
- **claimed** (*datetime*) – The time the item was claimed

- **owner_email** (*str*) – The email of the person claiming the item

**Returns**
A new *Item* instance representing the claimed item

**Return type**
*Item*

**static create_found_item**(*name: str*, *category:* Category, *found: datetime*, *location: str*, *finder_email: str*) → *Item*

Create a new found item.

**Parameters**

- **name** (*str*) – The name of the item
- **category** (Category) – The category of the item
- **found** (*datetime*) – The time the item was found
- **location** (*str*) – The location where the item was found
- **finder_email** (*str*) – The email of the person who found the item

**Returns**
A new *Item* instance representing the found item

**Return type**
*Item*

**static create_lost_item**(*name: str*, *category:* Category, *lost: datetime*, *owner_email: str*) → *Item*

Create a new lost item.

**Parameters**

- **name** (*str*) – The name of the item
- **category** (Category) – The category of the item
- **lost** (*datetime*) – The time the item was lost
- **owner_email** (*str*) – The email of the item's owner

**Returns**
A new *Item* instance representing the lost item

**Return type**
*Item*

**get**(*id:* EntityId) → *Entity* | None

Recursively find an entity with *id* in this subtree, or *None*.

**mark_as_found**(*found: datetime*, *location: str*, *finder_email: str*) → *Item*

Mark an item as found. Item must not have been found already.

**Parameters**

- **self** – The item to mark as found
- **found** (*datetime*) – The time the item was found
- **location** (*str*) – The location where the item was found
- **finder_email** (*str*) – The email of the person who found the item

**Returns**
A new *Item* instance representing the found item

**Return type**
*Item*

### lost_and_found.state.model

### Classes

| | |
|---|---|
| *AppModel*(hierarchy, entity) | |
| *ItemsModel*(hierarchy, entity) | Model for *Item* collections including a searchable view. |
| *ListModel*(hierarchy, entity) | Model helpers for list-like entities exposing *items* and mutations. |
| *Model*(hierarchy, entity) | Base model providing property and observe helpers for an entity. |
| *SearchParamsModel*(hierarchy, entity) | Model exposing *SearchParams* fields as properties. |

### lost_and_found.state.model.AppModel

**class** lost_and_found.state.model.**AppModel**(*hierarchy:* Hierarchy, *entity:* ValueObservable)

Bases: *Model*[*App*]

**__init__**(*hierarchy:* Hierarchy, *entity:* ValueObservable) → None

#### Methods

| | |
|---|---|
| *__init__*(hierarchy, entity) | |
| *observe*(getter) | Return a *ValueObservable* view for a derived value on the entity. |
| *property*(getter, setter) | Expose a *Property* backed by a getter/setter pair on the entity. |
| *update*(update) | Apply a transformation to the current entity and update the hierarchy. |

#### Attributes

| |
|---|
| items |
| search_params |

**observe**(*getter: Callable[[T], TValue]*) → *ValueObservable*

Return a *ValueObservable* view for a derived value on the entity.

**property**(*getter: Callable[[T], TValue]*, *setter: Callable[[T, TValue], T]*) → *Property*

Expose a *Property* backed by a getter/setter pair on the entity.

The returned *Property* is kept in sync with the underlying entity and updates propagate back into the hierarchy via *setter*.

**update**(*update: Callable[[T], T]*) → None

Apply a transformation to the current entity and update the hierarchy.

### lost_and_found.state.model.ItemsModel

**class** lost_and_found.state.model.**ItemsModel**(*hierarchy:* Hierarchy, *entity:* ValueObservable)

Bases: *ListModel*[*Item*]

Model for *Item* collections including a searchable view.

**__init__**(*hierarchy:* Hierarchy, *entity:* ValueObservable) → None

#### Methods

| | |
|---|---|
| *__init__*(hierarchy, entity) | |
| append(item) | |
| *observe*(getter) | Return a *ValueObservable* view for a derived value on the entity. |
| *property*(getter, setter) | Expose a *Property* backed by a getter/setter pair on the entity. |
| remove_all(to_remove) | |
| search(params) | |
| set(items) | |
| *update*(update) | Apply a transformation to the current entity and update the hierarchy. |

#### Attributes

| |
|---|
| items |

**observe**(*getter: Callable[[T], TValue]*) → *ValueObservable*

Return a *ValueObservable* view for a derived value on the entity.

**property**(*getter: Callable[[T], TValue]*, *setter: Callable[[T, TValue], T]*) → *Property*

Expose a *Property* backed by a getter/setter pair on the entity.

The returned *Property* is kept in sync with the underlying entity and updates propagate back into the hierarchy via *setter*.

**update**(*update: Callable[[T], T]*) → None

Apply a transformation to the current entity and update the hierarchy.

### lost_and_found.state.model.ListModel

**class** lost_and_found.state.model.**ListModel**(*hierarchy:* Hierarchy, *entity:* ValueObservable)

Bases: *Model*[*ListEntity*], Generic

Model helpers for list-like entities exposing *items* and mutations.

**__init__**(*hierarchy:* Hierarchy, *entity:* ValueObservable) → None

#### Methods

| |
|---|
| *__init__*(hierarchy, entity) |

continues on next page

Table 53 – continued from previous page

| | |
|---|---|
| append(item) | |
| *observe*(getter) | Return a *ValueObservable* view for a derived value on the entity. |
| *property*(getter, setter) | Expose a *Property* backed by a getter/setter pair on the entity. |
| remove_all(to_remove) | |
| set(items) | |
| *update*(update) | Apply a transformation to the current entity and update the hierarchy. |

### Attributes

| |
|---|
| items |

**observe**(*getter: Callable[[T], TValue]*) → *ValueObservable*

Return a *ValueObservable* view for a derived value on the entity.

**property**(*getter: Callable[[T], TValue]*, *setter: Callable[[T, TValue], T]*) → *Property*

Expose a *Property* backed by a getter/setter pair on the entity.

The returned *Property* is kept in sync with the underlying entity and updates propagate back into the hierarchy via *setter*.

**update**(*update: Callable[[T], T]*) → None

Apply a transformation to the current entity and update the hierarchy.

## lost_and_found.state.model.Model

**class** lost_and_found.state.model.**Model**(*hierarchy:* Hierarchy, *entity:* ValueObservable)

Bases: ABC, Generic

Base model providing property and observe helpers for an entity.

**__init__**(*hierarchy:* Hierarchy, *entity:* ValueObservable) → None

### Methods

| | |
|---|---|
| *__init__*(hierarchy, entity) | |
| *observe*(getter) | Return a *ValueObservable* view for a derived value on the entity. |
| *property*(getter, setter) | Expose a *Property* backed by a getter/setter pair on the entity. |
| *update*(update) | Apply a transformation to the current entity and update the hierarchy. |

**observe**(*getter: Callable[[T], TValue]*) → *ValueObservable*

Return a *ValueObservable* view for a derived value on the entity.

**property**(*getter: Callable[[T], TValue]*, *setter: Callable[[T, TValue], T]*) → *Property*

Expose a *Property* backed by a getter/setter pair on the entity.

The returned *Property* is kept in sync with the underlying entity and updates propagate back into the hierarchy via *setter*.

**update**(*update: Callable[[T], T]*) → None

Apply a transformation to the current entity and update the hierarchy.

### lost_and_found.state.model.SearchParamsModel

**class** lost_and_found.state.model.**SearchParamsModel**(*hierarchy:* Hierarchy, *entity:* ValueObservable)

Bases: *Model*[*SearchParams*]

Model exposing *SearchParams* fields as properties.

**__init__**(*hierarchy:* Hierarchy, *entity:* ValueObservable) → None

#### Methods

| | |
|---|---|
| *__init__*(hierarchy, entity) | |
| *observe*(getter) | Return a *ValueObservable* view for a derived value on the entity. |
| *property*(getter, setter) | Expose a *Property* backed by a getter/setter pair on the entity. |
| *update*(update) | Apply a transformation to the current entity and update the hierarchy. |

#### Attributes

| |
|---|
| category |
| name |

**observe**(*getter: Callable[[T], TValue]*) → *ValueObservable*

Return a *ValueObservable* view for a derived value on the entity.

**property**(*getter: Callable[[T], TValue]*, *setter: Callable[[T, TValue], T]*) → *Property*

Expose a *Property* backed by a getter/setter pair on the entity.

The returned *Property* is kept in sync with the underlying entity and updates propagate back into the hierarchy via *setter*.

**update**(*update: Callable[[T], T]*) → None

Apply a transformation to the current entity and update the hierarchy.

### lost_and_found.state.search

#### Classes

| | |
|---|---|
| *SearchParams*(id, children, name, category) | Entity storing the current search *name* and optional *category*. |

#### lost_and_found.state.search.SearchParams

**class** lost_and_found.state.search.**SearchParams**(*id:* EntityId, *children:* ImmutableList[Entity], *name: str*, *category:* Category | *None*)

Bases: *Entity*

Entity storing the current search *name* and optional *category*.

**__init__**(*id:* EntityId, *children:* ImmutableList[Entity], *name: str*, *category:* Category | *None*) → None

##### Methods

| | |
|---|---|
| *__init__*(id, children, name, category) | |
| *get*(id) | Recursively find an entity with *id* in this subtree, or *None*. |
| new() | |
| update_category(category) | |
| update_child(child) | |
| update_name(name) | |

##### Attributes

| |
|---|
| name |
| category |
| id |
| children |

**get**(*id:* EntityId) → *Entity* | None

Recursively find an entity with *id* in this subtree, or *None*.

### 4.1.5 lost_and_found.ui

#### Classes

| |
|---|
| *AppViewModel*(app) |

#### lost_and_found.ui.AppViewModel

**class** lost_and_found.ui.**AppViewModel**(*app:* AppModel)

Bases: *VerticalListViewModel*

**__init__**(*app:* AppModel)

##### Methods

| |
|---|
| *__init__*(app) |
| draw(parent) |

### Modules

| | |
|---|---|
| *add_found_item* | View model for adding a found item via a dialog. |
| *add_lost_item* | View model for adding a lost item via a dialog. |
| *claim* | Dialog view-model for claiming items. |
| *items* | View models composing the main items UI. |
| *mark_as_found* | Dialog view-model for marking multiple items as found. |
| *search_params* | View model exposing search parameter controls. |
| *widgets* | |

#### lost_and_found.ui.add_found_item

View model for adding a found item via a dialog.

### Classes

| | |
|---|---|
| *AddFoundItemViewModel*(items) | Dialog view-model for creating a new found *Item*. |

#### lost_and_found.ui.add_found_item.AddFoundItemViewModel

**class** lost_and_found.ui.add_found_item.**AddFoundItemViewModel**(*items:* ItemsModel)

Bases: *DialogViewModel*

Dialog view-model for creating a new found *Item*.

**__init__**(*items:* ItemsModel) → None

### Methods

| | |
|---|---|
| *__init__*(items) | |
| apply() | |
| draw(parent) | |
| validate() | |

#### lost_and_found.ui.add_lost_item

View model for adding a lost item via a dialog.

### Classes

| | |
|---|---|
| *AddLostItemViewModel*(items) | Dialog view-model for creating a new lost *Item*. |

#### lost_and_found.ui.add_lost_item.AddLostItemViewModel

**class** lost_and_found.ui.add_lost_item.**AddLostItemViewModel**(*items:* ItemsModel)

Bases: *DialogViewModel*

Dialog view-model for creating a new lost *Item*.

Exposes *Property* objects for the form fields and implements *validate*/*apply* for the dialog lifecycle.

**__init__**(*items:* ItemsModel) → None

#### Methods

| | |
|---|---|
| *__init__*(items) | |
| apply() | |
| draw(parent) | |
| validate() | |

### lost_and_found.ui.claim

Dialog view-model for claiming items.

#### Classes

| | |
|---|---|
| *ClaimItemsViewModel*(hierarchy, items) | Collect owner contact details and mark provided items as claimed. |

### lost_and_found.ui.claim.ClaimItemsViewModel

**class** lost_and_found.ui.claim.**ClaimItemsViewModel**(*hierarchy:* Hierarchy*[*Entity*]*, *items:* ImmutableList*[*Item*]*)

Bases: *DialogViewModel*

Collect owner contact details and mark provided items as claimed.

**__init__**(*hierarchy:* Hierarchy*[*Entity*]*, *items:* ImmutableList*[*Item*]*) → None

#### Methods

| | |
|---|---|
| *__init__*(hierarchy, items) | |
| apply() | |
| draw(parent) | |
| validate() | |

### lost_and_found.ui.items

View models composing the main items UI.

This module wires smaller view-model widgets into the top-level *ItemsViewModel* used by the UI to present and operate on *Item* entities.

#### Classes

| | |
|---|---|
| *ItemsViewModel*(items, search_params) | Composes table and action button view models for the items screen. |

### lost_and_found.ui.items.ItemsViewModel

**class** lost_and_found.ui.items.**ItemsViewModel**(*items:* ItemsModel, *search_params:* SearchParamsModel)

Bases: *VerticalListViewModel*

Composes table and action button view models for the items screen.

**__init__**(*items:* ItemsModel, *search_params:* SearchParamsModel) → None

#### Methods

| | |
|---|---|
| *__init__*(items, search_params) | |
| draw(parent) | |

### lost_and_found.ui.mark_as_found

Dialog view-model for marking multiple items as found.

#### Classes

| | |
|---|---|
| *MarkItemsAsFoundViewModel*(hierarchy, items) | Collects location and contact info and marks provided items as found. |

### lost_and_found.ui.mark_as_found.MarkItemsAsFoundViewModel

**class** lost_and_found.ui.mark_as_found.**MarkItemsAsFoundViewModel**(*hierarchy:* Hierarchy*[*Entity*]*, *items:* ImmutableList*[*Item*]*)

Bases: *DialogViewModel*

Collects location and contact info and marks provided items as found.

**__init__**(*hierarchy:* Hierarchy*[*Entity*]*, *items:* ImmutableList*[*Item*]*) → None

#### Methods

| | |
|---|---|
| *__init__*(hierarchy, items) | |
| apply() | |
| draw(parent) | |
| validate() | |

### lost_and_found.ui.search_params

View model exposing search parameter controls.

#### Classes

| | |
|---|---|
| *SearchParamsViewModel*(search_params) | View-model providing name and category controls for searching. |

### lost_and_found.ui.search_params.SearchParamsViewModel

**class** lost_and_found.ui.search_params.**SearchParamsViewModel**(*search_params:* SearchParamsModel)

Bases: *HorizontalListViewModel*

View-model providing name and category controls for searching.

**__init__**(*search_params:* SearchParamsModel) → None

#### Methods

| | |
|---|---|
| *__init__*(search_params) | |
| draw(parent) | |

### lost_and_found.ui.widgets

#### Modules

| | |
|---|---|
| *base* | Basic view and view-model base classes for UI widgets. |
| *button* | Button widget view-model and view. |
| *dialog* | Dialog wdiget view and view-model. |
| *entry* | Text entry view and view-model binding to a *Property*. |
| *grid* | Grid layout helpers for composing views in rows/columns. |
| *hlist* | Horizontal list helpers for arranging children left-to-right. |
| *label* | Simple label view and view-model. |
| *option_menu* | Option menu (dropdown) binding to a *Property* with formatting. |
| *table* | Table view backed by a *ValueObservable* of rows. |
| *vlist* | Vertical list helpers for stacking child view-models top-to-bottom. |

### lost_and_found.ui.widgets.base

Basic view and view-model base classes for UI widgets.

#### Classes

| | |
|---|---|
| *View*(vm) | Base class for views that render a *ViewModel*. |
| *ViewModel*(view) | Base class for view-models that hold presentation logic. |

### lost_and_found.ui.widgets.base.View

**class** lost_and_found.ui.widgets.base.**View**(*vm: T*)

Bases: ABC, Generic

Base class for views that render a *ViewModel*.

Subclasses should implement *draw* to construct tkinter widgets under a provided parent and return the created widget.

**__init__**(*vm: T*) → None

#### Methods

| | |
|---|---|
| *__init__*(vm) | |
| draw(parent) | |

### lost_and_found.ui.widgets.base.ViewModel

**class** lost_and_found.ui.widgets.base.**ViewModel**(*view:* View)

Bases: ABC

Base class for view-models that hold presentation logic.

**__init__**(*view:* View)

#### Methods

| | |
|---|---|
| *__init__*(view) | |
| draw(parent) | |

### lost_and_found.ui.widgets.button

Button widget view-model and view.

#### Classes

| | |
|---|---|
| *ButtonView*(vm) | Render a tkinter *Button* wired to the view-model. |
| *ButtonViewModel*(text[, enabled]) | View-model representing a button and its enabled state. |

### lost_and_found.ui.widgets.button.ButtonView

**class** lost_and_found.ui.widgets.button.**ButtonView**(*vm: T*)

Bases: *View*[*ButtonViewModel*]

Render a tkinter *Button* wired to the view-model.

**__init__**(*vm: T*) → None

#### Methods

| | |
|---|---|
| *__init__*(vm) | |
| draw(parent) | |

### lost_and_found.ui.widgets.button.ButtonViewModel

class lost_and_found.ui.widgets.button.**ButtonViewModel**(*text: str*, *enabled:* ValueObservable*[bool]* | *None = None*)

Bases: *ViewModel*

View-model representing a button and its enabled state.

**__init__**(*text: str*, *enabled:* ValueObservable*[bool]* | *None = None*) → None

### Methods

| | |
|---|---|
| *__init__*(text[, enabled]) | |
| draw(parent) | |

### Attributes

| | |
|---|---|
| *on_click* | Observable that emits when the button is clicked. |

**property on_click:** *Observable*[tuple[()]]

Observable that emits when the button is clicked.

## lost_and_found.ui.widgets.dialog

Dialog wdiget view and view-model.

### Classes

| | |
|---|---|
| *DialogView*(vm) | Render a simple modal dialog that delegates to a *DialogViewModel*. |
| *DialogViewModel*(title, body) | Base class for dialog view-models supporting validation/apply. |

## lost_and_found.ui.widgets.dialog.DialogView

class lost_and_found.ui.widgets.dialog.**DialogView**(*vm:* DialogViewModel)

Bases: Dialog, *View*[*DialogViewModel*]

Render a simple modal dialog that delegates to a *DialogViewModel*.

**__init__**(*vm:* DialogViewModel) → None

Initialize a dialog.

Arguments:

parent – a parent window (the application window)

title – the dialog title

### Methods

| | |
|---|---|
| *__init__*(vm) | Initialize a dialog. |

continues on next page

Table 85 – continued from previous page

| | |
|---|---|
| after(ms[, func]) | Call function once after given time. |
| after_cancel(id) | Cancel scheduling of function identified with ID. |
| after_idle(func, *args) | Call FUNC once if the Tcl main loop has no event to process. |
| anchor([anchor]) | The anchor value controls how to place the grid within the master when no row/column has any weight. |
| apply() | process the data |
| aspect([minNumer, minDenom, maxNumer, maxDenom]) | Instruct the window manager to set the aspect ratio (width/height) of this widget to be between MINNUMER/MINDENOM and MAXNUMER/MAXDENOM. |
| attributes(*args) | This subcommand returns or sets platform specific attributes |
| bbox([column, row, col2, row2]) | Return a tuple of integer coordinates for the bounding box of this widget controlled by the geometry manager grid. |
| bell([displayof]) | Ring a display's bell. |
| bind([sequence, func, add]) | Bind to this widget at event SEQUENCE a call to function FUNC. |
| bind_all([sequence, func, add]) | Bind to all widgets at an event SEQUENCE a call to function FUNC. |
| bind_class(className[, sequence, func, add]) | Bind to widgets with bindtag CLASSNAME at event SEQUENCE a call of function FUNC. |
| bindtags([tagList]) | Set or get the list of bindtags for this widget. |
| body(master) | create dialog body. |
| buttonbox() | add standard button box. |
| cancel([event]) | |
| cget(key) | Return the resource value for a KEY given as string. |
| client([name]) | Store NAME in WM_CLIENT_MACHINE property of this widget. |
| clipboard_append(string, **kw) | Append STRING to the Tk clipboard. |
| clipboard_clear(**kw) | Clear the data in the Tk clipboard. |
| clipboard_get(**kw) | Retrieve data from the clipboard on window's display. |
| colormapwindows(*wlist) | Store list of window names (WLIST) into WM_COLORMAPWINDOWS property of this widget. |
| columnconfigure(index[, cnf]) | Configure column INDEX of a grid. |
| command([value]) | Store VALUE in WM_COMMAND property. |
| config([cnf]) | Configure resources of a widget. |
| configure([cnf]) | Configure resources of a widget. |
| deiconify() | Deiconify this widget. |
| deletecommand(name) | Internal function. |
| destroy() | Destroy the window |
| draw(parent) | |
| event_add(virtual, *sequences) | Bind a virtual event VIRTUAL (of the form <<Name>>) to an event SEQUENCE such that the virtual event is triggered whenever SEQUENCE occurs. |
| event_delete(virtual, *sequences) | Unbind a virtual event VIRTUAL from SEQUENCE. |
| event_generate(sequence, **kw) | Generate an event SEQUENCE. |

continues on next page

Table 85 – continued from previous page

| | |
|---|---|
| event_info([virtual]) | Return a list of all virtual events or the information about the SEQUENCE bound to the virtual event VIRTUAL. |
| focus() | Direct input focus to this widget. |
| focus_displayof() | Return the widget which has currently the focus on the display where this widget is located. |
| focus_force() | Direct input focus to this widget even if the application does not have the focus. |
| focus_get() | Return the widget which has currently the focus in the application. |
| focus_lastfor() | Return the widget which would have the focus if top level for this widget gets the focus from the window manager. |
| focus_set() | Direct input focus to this widget. |
| focusmodel([model]) | Set focus model to MODEL. |
| forget(window) | The window will be unmapped from the screen and will no longer be managed by wm. |
| frame() | Return identifier for decorative frame of this widget if present. |
| geometry([newGeometry]) | Set geometry to NEWGEOMETRY of the form =widthxheight+x+y. |
| getboolean(s) | Return a boolean value for Tcl boolean values true and false given as parameter. |
| getdouble(s) | |
| getint(s) | |
| getvar([name]) | Return value of Tcl variable NAME. |
| grab_current() | Return widget which has currently the grab in this application or None. |
| grab_release() | Release grab for this widget if currently set. |
| grab_set() | Set grab for this widget. |
| grab_set_global() | Set global grab for this widget. |
| grab_status() | Return None, "local" or "global" if this widget has no, a local or a global grab. |
| grid([baseWidth, baseHeight, widthInc, ...]) | Instruct the window manager that this widget shall only be resized on grid boundaries. |
| grid_anchor([anchor]) | The anchor value controls how to place the grid within the master when no row/column has any weight. |
| grid_bbox([column, row, col2, row2]) | Return a tuple of integer coordinates for the bounding box of this widget controlled by the geometry manager grid. |
| grid_columnconfigure(index[, cnf]) | Configure column INDEX of a grid. |
| grid_location(x, y) | Return a tuple of column and row which identify the cell at which the pixel at position X and Y inside the master widget is located. |
| grid_propagate([flag]) | Set or get the status for propagation of geometry information. |
| grid_rowconfigure(index[, cnf]) | Configure row INDEX of a grid. |
| grid_size() | Return a tuple of the number of column and rows in the grid. |
| grid_slaves([row, column]) | Return a list of all slaves of this widget in its packing order. |

continues on next page

Table 85 – continued from previous page

| | |
|---|---|
| group([pathName]) | Set the group leader widgets for related widgets to PATHNAME. |
| iconbitmap([bitmap, default]) | Set bitmap for the iconified widget to BITMAP. |
| iconify() | Display widget as icon. |
| iconmask([bitmap]) | Set mask for the icon bitmap of this widget. |
| iconname([newName]) | Set the name of the icon for this widget. |
| iconphoto([default]) | Sets the titlebar icon for this window based on the named photo images passed through args. |
| iconposition([x, y]) | Set the position of the icon of this widget to X and Y. |
| iconwindow([pathName]) | Set widget PATHNAME to be displayed instead of icon. |
| image_names() | Return a list of all existing image names. |
| image_types() | Return a list of all available image types (e.g. photo bitmap). |
| info_patchlevel() | Returns the exact version of the Tcl library. |
| keys() | Return a list of all resource names of this widget. |
| lift([aboveThis]) | Raise this widget in the stacking order. |
| lower([belowThis]) | Lower this widget in the stacking order. |
| mainloop([n]) | Call the mainloop of Tk. |
| manage(widget) | The widget specified will become a stand alone top-level window. |
| maxsize([width, height]) | Set max WIDTH and HEIGHT for this widget. |
| minsize([width, height]) | Set min WIDTH and HEIGHT for this widget. |
| nametowidget(name) | Return the Tkinter instance of a widget identified by its Tcl name NAME. |
| ok([event]) | |
| option_add(pattern, value[, priority]) | Set a VALUE (second parameter) for an option PATTERN (first parameter). |
| option_clear() | Clear the option database. |
| option_get(name, className) | Return the value for an option NAME for this widget with CLASSNAME. |
| option_readfile(fileName[, priority]) | Read file FILENAME into the option database. |
| overrideredirect([boolean]) | Instruct the window manager to ignore this widget if BOOLEAN is given with 1. |
| pack_propagate([flag]) | Set or get the status for propagation of geometry information. |
| pack_slaves() | Return a list of all slaves of this widget in its packing order. |
| place_slaves() | Return a list of all slaves of this widget in its packing order. |
| positionfrom([who]) | Instruct the window manager that the position of this widget shall be defined by the user if WHO is "user", and by its own policy if WHO is "program". |
| propagate([flag]) | Set or get the status for propagation of geometry information. |
| protocol([name, func]) | Bind function FUNC to command NAME for this widget. |
| quit() | Quit the Tcl interpreter. |
| register(func[, subst, needcleanup]) | Return a newly created Tcl function. |
| resizable([width, height]) | Instruct the window manager whether this width can be resized in WIDTH or HEIGHT. |
| rowconfigure(index[, cnf]) | Configure row INDEX of a grid. |

continues on next page

Table 85 – continued from previous page

| | |
|---|---|
| selection_clear(**kw) | Clear the current X selection. |
| selection_get(**kw) | Return the contents of the current X selection. |
| selection_handle(command, **kw) | Specify a function COMMAND to call if the X selection owned by this widget is queried by another application. |
| selection_own(**kw) | Become owner of X selection. |
| selection_own_get(**kw) | Return owner of X selection. |
| send(interp, cmd, *args) | Send Tcl command CMD to different interpreter INTERP to be executed. |
| setvar([name, value]) | Set Tcl variable NAME to VALUE. |
| size() | Return a tuple of the number of column and rows in the grid. |
| sizefrom([who]) | Instruct the window manager that the size of this widget shall be defined by the user if WHO is "user", and by its own policy if WHO is "program". |
| slaves() | Return a list of all slaves of this widget in its packing order. |
| state([newstate]) | Query or set the state of this widget as one of normal, icon, iconic (see wm_iconwindow), withdrawn, or zoomed (Windows only). |
| title([string]) | Set the title of this widget. |
| tk_bisque() | Change the color scheme to light brown as used in Tk 3.6 and before. |
| tk_focusFollowsMouse() | The widget under mouse will get automatically focus. |
| tk_focusNext() | Return the next widget in the focus order which follows widget which has currently the focus. |
| tk_focusPrev() | Return previous widget in the focus order. |
| tk_setPalette(*args, **kw) | Set a new color scheme for all widget elements. |
| tk_strictMotif([boolean]) | Set Tcl internal variable, whether the look and feel should adhere to Motif. |
| tkraise([aboveThis]) | Raise this widget in the stacking order. |
| transient([master]) | Instruct the window manager that this widget is transient with regard to widget MASTER. |
| unbind(sequence[, funcid]) | Unbind for this widget the event SEQUENCE. |
| unbind_all(sequence) | Unbind for all widgets for event SEQUENCE all functions. |
| unbind_class(className, sequence) | Unbind for all widgets with bindtag CLASSNAME for event SEQUENCE all functions. |
| update() | Enter event loop until all pending events have been processed by Tcl. |
| update_idletasks() | Enter event loop until all idle callbacks have been called. |
| validate() | validate the data |
| wait_variable([name]) | Wait until the variable is modified. |
| wait_visibility([window]) | Wait until the visibility of a WIDGET changes (e.g. it appears). |
| wait_window([window]) | Wait until a WIDGET is destroyed. |
| waitvar([name]) | Wait until the variable is modified. |
| winfo_atom(name[, displayof]) | Return integer which represents atom NAME. |
| winfo_atomname(id[, displayof]) | Return name of atom with identifier ID. |
| winfo_cells() | Return number of cells in the colormap for this widget. |

continues on next page

Table 85 – continued from previous page

| | |
|---|---|
| *winfo_children*() | Return a list of all widgets which are children of this widget. |
| *winfo_class*() | Return window class name of this widget. |
| *winfo_colormapfull*() | Return True if at the last color request the colormap was full. |
| *winfo_containing*(rootX, rootY[, displayof]) | Return the widget which is at the root coordinates ROOTX, ROOTY. |
| *winfo_depth*() | Return the number of bits per pixel. |
| *winfo_exists*() | Return true if this widget exists. |
| *winfo_fpixels*(number) | Return the number of pixels for the given distance NUMBER (e.g. "3c") as float. |
| *winfo_geometry*() | Return geometry string for this widget in the form "widthxheight+X+Y". |
| *winfo_height*() | Return height of this widget. |
| *winfo_id*() | Return identifier ID for this widget. |
| *winfo_interps*([displayof]) | Return the name of all Tcl interpreters for this display. |
| *winfo_ismapped*() | Return true if this widget is mapped. |
| *winfo_manager*() | Return the window manager name for this widget. |
| *winfo_name*() | Return the name of this widget. |
| *winfo_parent*() | Return the name of the parent of this widget. |
| *winfo_pathname*(id[, displayof]) | Return the pathname of the widget given by ID. |
| *winfo_pixels*(number) | Rounded integer value of winfo_fpixels. |
| *winfo_pointerx*() | Return the x coordinate of the pointer on the root window. |
| *winfo_pointerxy*() | Return a tuple of x and y coordinates of the pointer on the root window. |
| *winfo_pointery*() | Return the y coordinate of the pointer on the root window. |
| *winfo_reqheight*() | Return requested height of this widget. |
| *winfo_reqwidth*() | Return requested width of this widget. |
| *winfo_rgb*(color) | Return a tuple of integer RGB values in range(65536) for color in this widget. |
| *winfo_rootx*() | Return x coordinate of upper left corner of this widget on the root window. |
| *winfo_rooty*() | Return y coordinate of upper left corner of this widget on the root window. |
| *winfo_screen*() | Return the screen name of this widget. |
| *winfo_screencells*() | Return the number of the cells in the colormap of the screen of this widget. |
| *winfo_screendepth*() | Return the number of bits per pixel of the root window of the screen of this widget. |
| *winfo_screenheight*() | Return the number of pixels of the height of the screen of this widget in pixel. |
| *winfo_screenmmheight*() | Return the number of pixels of the height of the screen of this widget in mm. |
| *winfo_screenmmwidth*() | Return the number of pixels of the width of the screen of this widget in mm. |
| *winfo_screenvisual*() | Return one of the strings directcolor, grayscale, pseudocolor, staticcolor, staticgray, or truecolor for the default colormodel of this screen. |
| *winfo_screenwidth*() | Return the number of pixels of the width of the screen of this widget in pixel. |

continues on next page

Table 85 – continued from previous page

| | |
|---|---|
| winfo_server() | Return information of the X-Server of the screen of this widget in the form "XmajorRminor vendor vendorVersion". |
| winfo_toplevel() | Return the toplevel widget of this widget. |
| winfo_viewable() | Return true if the widget and all its higher ancestors are mapped. |
| winfo_visual() | Return one of the strings directcolor, grayscale, pseudocolor, staticcolor, staticgray, or truecolor for the colormodel of this widget. |
| winfo_visualid() | Return the X identifier for the visual for this widget. |
| winfo_visualsavailable([includeids]) | Return a list of all visuals available for the screen of this widget. |
| winfo_vrootheight() | Return the height of the virtual root window associated with this widget in pixels. |
| winfo_vrootwidth() | Return the width of the virtual root window associated with this widget in pixel. |
| winfo_vrootx() | Return the x offset of the virtual root relative to the root window of the screen of this widget. |
| winfo_vrooty() | Return the y offset of the virtual root relative to the root window of the screen of this widget. |
| winfo_width() | Return the width of this widget. |
| winfo_x() | Return the x coordinate of the upper left corner of this widget in the parent. |
| winfo_y() | Return the y coordinate of the upper left corner of this widget in the parent. |
| withdraw() | Withdraw this widget from the screen such that it is unmapped and forgotten by the window manager. |
| wm_aspect([minNumer, minDenom, maxNumer, ...]) | Instruct the window manager to set the aspect ratio (width/height) of this widget to be between MINNUMER/MINDENOM and MAXNUMER/MAXDENOM. |
| wm_attributes(*args) | This subcommand returns or sets platform specific attributes |
| wm_client([name]) | Store NAME in WM_CLIENT_MACHINE property of this widget. |
| wm_colormapwindows(*wlist) | Store list of window names (WLIST) into WM_COLORMAPWINDOWS property of this widget. |
| wm_command([value]) | Store VALUE in WM_COMMAND property. |
| wm_deiconify() | Deiconify this widget. |
| wm_focusmodel([model]) | Set focus model to MODEL. |
| wm_forget(window) | The window will be unmapped from the screen and will no longer be managed by wm. |
| wm_frame() | Return identifier for decorative frame of this widget if present. |
| wm_geometry([newGeometry]) | Set geometry to NEWGEOMETRY of the form =widthxheight+x+y. |
| wm_grid([baseWidth, baseHeight, widthInc, ...]) | Instruct the window manager that this widget shall only be resized on grid boundaries. |
| wm_group([pathName]) | Set the group leader widgets for related widgets to PATHNAME. |
| wm_iconbitmap([bitmap, default]) | Set bitmap for the iconified widget to BITMAP. |

continues on next page

Table 85 – continued from previous page

| | |
|---|---|
| wm_iconify() | Display widget as icon. |
| wm_iconmask([bitmap]) | Set mask for the icon bitmap of this widget. |
| wm_iconname([newName]) | Set the name of the icon for this widget. |
| wm_iconphoto([default]) | Sets the titlebar icon for this window based on the named photo images passed through args. |
| wm_iconposition([x, y]) | Set the position of the icon of this widget to X and Y. |
| wm_iconwindow([pathName]) | Set widget PATHNAME to be displayed instead of icon. |
| wm_manage(widget) | The widget specified will become a stand alone top-level window. |
| wm_maxsize([width, height]) | Set max WIDTH and HEIGHT for this widget. |
| wm_minsize([width, height]) | Set min WIDTH and HEIGHT for this widget. |
| wm_overrideredirect([boolean]) | Instruct the window manager to ignore this widget if BOOLEAN is given with 1. |
| wm_positionfrom([who]) | Instruct the window manager that the position of this widget shall be defined by the user if WHO is "user", and by its own policy if WHO is "program". |
| wm_protocol([name, func]) | Bind function FUNC to command NAME for this widget. |
| wm_resizable([width, height]) | Instruct the window manager whether this width can be resized in WIDTH or HEIGHT. |
| wm_sizefrom([who]) | Instruct the window manager that the size of this widget shall be defined by the user if WHO is "user", and by its own policy if WHO is "program". |
| wm_state([newstate]) | Query or set the state of this widget as one of normal, icon, iconic (see wm_iconwindow), withdrawn, or zoomed (Windows only). |
| wm_title([string]) | Set the title of this widget. |
| wm_transient([master]) | Instruct the window manager that this widget is transient with regard to widget MASTER. |
| wm_withdraw() | Withdraw this widget from the screen such that it is unmapped and forgotten by the window manager. |

**after**(*ms*, *func=None*, **args*)

Call function once after given time.

MS specifies the time in milliseconds. FUNC gives the function which shall be called. Additional parameters are given as parameters to the function call. Return identifier to cancel scheduling with after_cancel.

**after_cancel**(*id*)

Cancel scheduling of function identified with ID.

Identifier returned by after or after_idle must be given as first parameter.

**after_idle**(*func*, **args*)

Call FUNC once if the Tcl main loop has no event to process.

Return an identifier to cancel the scheduling with after_cancel.

**anchor**(*anchor=None*)

The anchor value controls how to place the grid within the master when no row/column has any weight.

The default anchor is nw.

**apply**() → None

process the data

This method is called automatically to process the data, *after* the dialog is destroyed. By default, it does nothing.

**aspect**(*minNumer=None*, *minDenom=None*, *maxNumer=None*, *maxDenom=None*)

Instruct the window manager to set the aspect ratio (width/height) of this widget to be between MINNUMER/MINDENOM and MAXNUMER/MAXDENOM. Return a tuple of the actual values if no argument is given.

**attributes**(**args*)

This subcommand returns or sets platform specific attributes

The first form returns a list of the platform specific flags and their values. The second form returns the value for the specific option. The third form sets one or more of the values. The values are as follows:

On Windows, -disabled gets or sets whether the window is in a disabled state. -toolwindow gets or sets the style of the window to toolwindow (as defined in the MSDN). -topmost gets or sets whether this is a topmost window (displays above all other windows).

On Macintosh, XXXXX

On Unix, there are currently no special attribute values.

**bbox**(*column=None*, *row=None*, *col2=None*, *row2=None*)

Return a tuple of integer coordinates for the bounding box of this widget controlled by the geometry manager grid.

If COLUMN, ROW is given the bounding box applies from the cell with row and column 0 to the specified cell. If COL2 and ROW2 are given the bounding box starts at that cell.

The returned integers specify the offset of the upper left corner in the master widget and the width and height.

**bell**(*displayof=0*)

Ring a display's bell.

**bind**(*sequence=None*, *func=None*, *add=None*)

Bind to this widget at event SEQUENCE a call to function FUNC.

SEQUENCE is a string of concatenated event patterns. An event pattern is of the form <MODIFIER-MODIFIER-TYPE-DETAIL> where MODIFIER is one of Control, Mod2, M2, Shift, Mod3, M3, Lock, Mod4, M4, Button1, B1, Mod5, M5 Button2, B2, Meta, M, Button3, B3, Alt, Button4, B4, Double, Button5, B5 Triple, Mod1, M1. TYPE is one of Activate, Enter, Map, ButtonPress, Button, Expose, Motion, ButtonRelease FocusIn, MouseWheel, Circulate, FocusOut, Property, Colormap, Gravity Reparent, Configure, KeyPress, Key, Unmap, Deactivate, KeyRelease Visibility, Destroy, Leave and DETAIL is the button number for ButtonPress, ButtonRelease and DETAIL is the Keysym for KeyPress and KeyRelease. Examples are <Control-Button-1> for pressing Control and mouse button 1 or <Alt-A> for pressing A and the Alt key (KeyPress can be omitted). An event pattern can also be a virtual event of the form <<AString>> where AString can be arbitrary. This event can be generated by event_generate. If events are concatenated they must appear shortly after each other.

FUNC will be called if the event sequence occurs with an instance of Event as argument. If the return value of FUNC is "break" no further bound function is invoked.

An additional boolean parameter ADD specifies whether FUNC will be called additionally to the other bound function or whether it will replace the previous function.

Bind will return an identifier to allow deletion of the bound function with unbind without memory leak.

If FUNC or SEQUENCE is omitted the bound function or list of bound events are returned.

**bind_all**(*sequence=None*, *func=None*, *add=None*)

Bind to all widgets at an event SEQUENCE a call to function FUNC. An additional boolean parameter ADD specifies whether FUNC will be called additionally to the other bound function or whether it will replace the previous function. See bind for the return value.

**bind_class**(*className*, *sequence=None*, *func=None*, *add=None*)

Bind to widgets with bindtag CLASSNAME at event SEQUENCE a call of function FUNC. An additional boolean parameter ADD specifies whether FUNC will be called additionally to the other bound function or whether it will replace the previous function. See bind for the return value.

**bindtags**(*tagList=None*)

Set or get the list of bindtags for this widget.

With no argument return the list of all bindtags associated with this widget. With a list of strings as argument the bindtags are set to this list. The bindtags determine in which order events are processed (see bind).

**body**(*master: Misc*) → Misc

create dialog body.

return widget that should have initial focus. This method should be overridden, and is called by the __init__ method.

**buttonbox**()

add standard button box.

override if you do not want the standard buttons

**cget**(*key*)

Return the resource value for a KEY given as string.

**client**(*name=None*)

Store NAME in WM_CLIENT_MACHINE property of this widget. Return current value.

**clipboard_append**(*string*, ***kw*)

Append STRING to the Tk clipboard.

A widget specified at the optional displayof keyword argument specifies the target display. The clipboard can be retrieved with selection_get.

**clipboard_clear**(***kw*)

Clear the data in the Tk clipboard.

A widget specified for the optional displayof keyword argument specifies the target display.

**clipboard_get**(***kw*)

Retrieve data from the clipboard on window's display.

The window keyword defaults to the root window of the Tkinter application.

The type keyword specifies the form in which the data is to be returned and should be an atom name such as STRING or FILE_NAME. Type defaults to STRING, except on X11, where the default is to try UTF8_STRING and fall back to STRING.

This command is equivalent to:

selection_get(CLIPBOARD)

**colormapwindows**(**wlist*)

Store list of window names (WLIST) into WM_COLORMAPWINDOWS property of this widget. This list contains windows whose colormaps differ from their parents. Return current list of widgets if WLIST is empty.

**columnconfigure**(*index*, *cnf={}*, ***kw*)

Configure column INDEX of a grid.

Valid resources are minsize (minimum size of the column), weight (how much does additional space propagate to this column) and pad (how much space to let additionally).

**command**(*value=None*)

Store VALUE in WM_COMMAND property. It is the command which shall be used to invoke the application. Return current command if VALUE is None.

**config**(*cnf=None*, ***kw*)

Configure resources of a widget.

The values for resources are specified as keyword arguments. To get an overview about the allowed keyword arguments call the method keys.

**configure**(*cnf=None*, ***kw*)

Configure resources of a widget.

The values for resources are specified as keyword arguments. To get an overview about the allowed keyword arguments call the method keys.

**deiconify**()

Deiconify this widget. If it was never mapped it will not be mapped. On Windows it will raise this widget and give it the focus.

**deletecommand**(*name*)

Internal function.

Delete the Tcl command provided in NAME.

**destroy**()

Destroy the window

**event_add**(*virtual*, **sequences*)

Bind a virtual event VIRTUAL (of the form <<Name>>) to an event SEQUENCE such that the virtual event is triggered whenever SEQUENCE occurs.

**event_delete**(*virtual*, **sequences*)

Unbind a virtual event VIRTUAL from SEQUENCE.

**event_generate**(*sequence*, ***kw*)

Generate an event SEQUENCE. Additional keyword arguments specify parameter of the event (e.g. x, y, rootx, rooty).

**event_info**(*virtual=None*)

Return a list of all virtual events or the information about the SEQUENCE bound to the virtual event VIRTUAL.

**focus**()

Direct input focus to this widget.

If the application currently does not have the focus this widget will get the focus if the application gets the focus through the window manager.

**focus_displayof**()

Return the widget which has currently the focus on the display where this widget is located.

Return None if the application does not have the focus.

**focus_force**()

Direct input focus to this widget even if the application does not have the focus. Use with caution!

**focus_get**()

Return the widget which has currently the focus in the application.

Use focus_displayof to allow working with several displays. Return None if application does not have the focus.

**focus_lastfor**()

Return the widget which would have the focus if top level for this widget gets the focus from the window manager.

**focus_set**()

Direct input focus to this widget.

If the application currently does not have the focus this widget will get the focus if the application gets the focus through the window manager.

**focusmodel**(*model=None*)

Set focus model to MODEL. "active" means that this widget will claim the focus itself, "passive" means that the window manager shall give the focus. Return current focus model if MODEL is None.

**forget**(*window*)

The window will be unmapped from the screen and will no longer be managed by wm. toplevel windows will be treated like frame windows once they are no longer managed by wm, however, the menu option configuration will be remembered and the menus will return once the widget is managed again.

**frame**()

Return identifier for decorative frame of this widget if present.

**geometry**(*newGeometry=None*)

Set geometry to NEWGEOMETRY of the form =widthxheight+x+y. Return current value if None is given.

**getboolean**(*s*)

Return a boolean value for Tcl boolean values true and false given as parameter.

**getvar**(*name='PY_VAR'*)

Return value of Tcl variable NAME.

**grab_current**()

Return widget which has currently the grab in this application or None.

**grab_release**()

Release grab for this widget if currently set.

**grab_set**()

Set grab for this widget.

A grab directs all events to this and descendant widgets in the application.

**grab_set_global**()

Set global grab for this widget.

A global grab directs all events to this and descendant widgets on the display. Use with caution - other applications do not get events anymore.

**grab_status**()

Return None, "local" or "global" if this widget has no, a local or a global grab.

**grid**(*baseWidth=None*, *baseHeight=None*, *widthInc=None*, *heightInc=None*)

Instruct the window manager that this widget shall only be resized on grid boundaries. WIDTHINC and HEIGHTINC are the width and height of a grid unit in pixels. BASEWIDTH and BASEHEIGHT are the number of grid units requested in Tk_GeometryRequest.

**grid_anchor**(*anchor=None*)

The anchor value controls how to place the grid within the master when no row/column has any weight.

The default anchor is nw.

**grid_bbox**(*column=None*, *row=None*, *col2=None*, *row2=None*)

Return a tuple of integer coordinates for the bounding box of this widget controlled by the geometry manager grid.

If COLUMN, ROW is given the bounding box applies from the cell with row and column 0 to the specified cell. If COL2 and ROW2 are given the bounding box starts at that cell.

The returned integers specify the offset of the upper left corner in the master widget and the width and height.

**grid_columnconfigure**(*index*, *cnf={}*, ***kw*)

Configure column INDEX of a grid.

Valid resources are minsize (minimum size of the column), weight (how much does additional space propagate to this column) and pad (how much space to let additionally).

**grid_location**(*x*, *y*)

Return a tuple of column and row which identify the cell at which the pixel at position X and Y inside the master widget is located.

**grid_propagate**(*flag=['_noarg_']*)

Set or get the status for propagation of geometry information.

A boolean argument specifies whether the geometry information of the slaves will determine the size of this widget. If no argument is given, the current setting will be returned.

**grid_rowconfigure**(*index*, *cnf={}*, ***kw*)

Configure row INDEX of a grid.

Valid resources are minsize (minimum size of the row), weight (how much does additional space propagate to this row) and pad (how much space to let additionally).

**grid_size**()

Return a tuple of the number of column and rows in the grid.

**grid_slaves**(*row=None*, *column=None*)

Return a list of all slaves of this widget in its packing order.

**group**(*pathName=None*)

Set the group leader widgets for related widgets to PATHNAME. Return the group leader of this widget if None is given.

**iconbitmap**(*bitmap=None*, *default=None*)

Set bitmap for the iconified widget to BITMAP. Return the bitmap if None is given.

Under Windows, the DEFAULT parameter can be used to set the icon for the widget and any descendants that don't have an icon set explicitly. DEFAULT can be the relative path to a .ico file (example: root.iconbitmap(default='myicon.ico') ). See Tk documentation for more information.

**iconify**()

Display widget as icon.

**iconmask**(*bitmap=None*)

Set mask for the icon bitmap of this widget. Return the mask if None is given.

**iconname**(*newName=None*)

Set the name of the icon for this widget. Return the name if None is given.

**iconphoto**(*default=False*, **args*)

Sets the titlebar icon for this window based on the named photo images passed through args. If default is True, this is applied to all future created toplevels as well.

The data in the images is taken as a snapshot at the time of invocation. If the images are later changed, this is not reflected to the titlebar icons. Multiple images are accepted to allow different images sizes to be provided. The window manager may scale provided icons to an appropriate size.

On Windows, the images are packed into a Windows icon structure. This will override an icon specified to wm_iconbitmap, and vice versa.

On X, the images are arranged into the _NET_WM_ICON X property, which most modern window managers support. An icon specified by wm_iconbitmap may exist simultaneously.

On Macintosh, this currently does nothing.

**iconposition**(*x=None*, *y=None*)

Set the position of the icon of this widget to X and Y. Return a tuple of the current values of X and X if None is given.

**iconwindow**(*pathName=None*)

Set widget PATHNAME to be displayed instead of icon. Return the current value if None is given.

**image_names**()

Return a list of all existing image names.

**image_types**()

Return a list of all available image types (e.g. photo bitmap).

**info_patchlevel**()

Returns the exact version of the Tcl library.

**keys**()

Return a list of all resource names of this widget.

**lift**(*aboveThis=None*)

Raise this widget in the stacking order.

**lower**(*belowThis=None*)

Lower this widget in the stacking order.

**mainloop**(*n=0*)

Call the mainloop of Tk.

**manage**(*widget*)

The widget specified will become a stand alone top-level window. The window will be decorated with the window managers title bar, etc.

**maxsize**(*width=None*, *height=None*)

Set max WIDTH and HEIGHT for this widget. If the window is gridded the values are given in grid units. Return the current values if None is given.

**minsize**(*width=None*, *height=None*)

Set min WIDTH and HEIGHT for this widget. If the window is gridded the values are given in grid units. Return the current values if None is given.

**nametowidget**(*name*)

Return the Tkinter instance of a widget identified by its Tcl name NAME.

**option_add**(*pattern*, *value*, *priority=None*)

Set a VALUE (second parameter) for an option PATTERN (first parameter).

An optional third parameter gives the numeric priority (defaults to 80).

**option_clear**()

Clear the option database.

It will be reloaded if option_add is called.

**option_get**(*name*, *className*)

Return the value for an option NAME for this widget with CLASSNAME.

Values with higher priority override lower values.

**option_readfile**(*fileName*, *priority=None*)

Read file FILENAME into the option database.

An optional second parameter gives the numeric priority.

**overrideredirect**(*boolean=None*)

Instruct the window manager to ignore this widget if BOOLEAN is given with 1. Return the current value if None is given.

**pack_propagate**(*flag=['_noarg_']*)

Set or get the status for propagation of geometry information.

A boolean argument specifies whether the geometry information of the slaves will determine the size of this widget. If no argument is given the current setting will be returned.

**pack_slaves**()

Return a list of all slaves of this widget in its packing order.

**place_slaves**()

Return a list of all slaves of this widget in its packing order.

**positionfrom**(*who=None*)

Instruct the window manager that the position of this widget shall be defined by the user if WHO is “user”, and by its own policy if WHO is “program”.

**propagate**(*flag=['_noarg_']*)

Set or get the status for propagation of geometry information.

A boolean argument specifies whether the geometry information of the slaves will determine the size of this widget. If no argument is given the current setting will be returned.

**protocol**(*name=None*, *func=None*)

Bind function FUNC to command NAME for this widget. Return the function bound to NAME if None is given. NAME could be e.g. "WM_SAVE_YOURSELF" or "WM_DELETE_WINDOW".

**quit**()

Quit the Tcl interpreter. All widgets will be destroyed.

**register**(*func*, *subst=None*, *needcleanup=1*)

Return a newly created Tcl function. If this function is called, the Python function FUNC will be executed. An optional function SUBST can be given which will be executed before FUNC.

**resizable**(*width=None*, *height=None*)

Instruct the window manager whether this width can be resized in WIDTH or HEIGHT. Both values are boolean values.

**rowconfigure**(*index*, *cnf={}*, ***kw*)

Configure row INDEX of a grid.

Valid resources are minsize (minimum size of the row), weight (how much does additional space propagate to this row) and pad (how much space to let additionally).

**selection_clear**(***kw*)

Clear the current X selection.

**selection_get**(***kw*)

Return the contents of the current X selection.

A keyword parameter selection specifies the name of the selection and defaults to PRIMARY. A keyword parameter displayof specifies a widget on the display to use. A keyword parameter type specifies the form of data to be fetched, defaulting to STRING except on X11, where UTF8_STRING is tried before STRING.

**selection_handle**(*command*, ***kw*)

Specify a function COMMAND to call if the X selection owned by this widget is queried by another application.

This function must return the contents of the selection. The function will be called with the arguments OFFSET and LENGTH which allows the chunking of very long selections. The following keyword parameters can be provided: selection - name of the selection (default PRIMARY), type - type of the selection (e.g. STRING, FILE_NAME).

**selection_own**(***kw*)

Become owner of X selection.

A keyword parameter selection specifies the name of the selection (default PRIMARY).

**selection_own_get**(***kw*)

Return owner of X selection.

The following keyword parameter can be provided: selection - name of the selection (default PRIMARY), type - type of the selection (e.g. STRING, FILE_NAME).

**send**(*interp*, *cmd*, **args*)

Send Tcl command CMD to different interpreter INTERP to be executed.

**setvar**(*name='PY_VAR'*, *value='1'*)

Set Tcl variable NAME to VALUE.

**size**()

Return a tuple of the number of column and rows in the grid.

**sizefrom**(*who=None*)

Instruct the window manager that the size of this widget shall be defined by the user if WHO is "user", and by its own policy if WHO is "program".

**slaves**()

Return a list of all slaves of this widget in its packing order.

**state**(*newstate=None*)

Query or set the state of this widget as one of normal, icon, iconic (see wm_iconwindow), withdrawn, or zoomed (Windows only).

**title**(*string=None*)

Set the title of this widget.

**tk_bisque**()

Change the color scheme to light brown as used in Tk 3.6 and before.

**tk_focusFollowsMouse**()

The widget under mouse will get automatically focus. Can not be disabled easily.

**tk_focusNext**()

Return the next widget in the focus order which follows widget which has currently the focus.

The focus order first goes to the next child, then to the children of the child recursively and then to the next sibling which is higher in the stacking order. A widget is omitted if it has the takefocus resource set to 0.

**tk_focusPrev**()

Return previous widget in the focus order. See tk_focusNext for details.

**tk_setPalette**(**args*, ***kw*)

Set a new color scheme for all widget elements.

A single color as argument will cause that all colors of Tk widget elements are derived from this. Alternatively several keyword parameters and its associated colors can be given. The following keywords are valid: activeBackground, foreground, selectColor, activeForeground, highlightBackground, selectBackground, background, highlightColor, selectForeground, disabledForeground, insertBackground, troughColor.

**tk_strictMotif**(*boolean=None*)

Set Tcl internal variable, whether the look and feel should adhere to Motif.

A parameter of 1 means adhere to Motif (e.g. no color change if mouse passes over slider). Returns the set value.

**tkraise**(*aboveThis=None*)

Raise this widget in the stacking order.

**transient**(*master=None*)

Instruct the window manager that this widget is transient with regard to widget MASTER.

**unbind**(*sequence*, *funcid=None*)

Unbind for this widget the event SEQUENCE.

If FUNCID is given, only unbind the function identified with FUNCID and also delete the corresponding Tcl command.

Otherwise destroy the current binding for SEQUENCE, leaving SEQUENCE unbound.

**unbind_all**(*sequence*)

Unbind for all widgets for event SEQUENCE all functions.

**unbind_class**(*className*, *sequence*)

Unbind for all widgets with bindtag CLASSNAME for event SEQUENCE all functions.

**update**()

Enter event loop until all pending events have been processed by Tcl.

**update_idletasks**()

Enter event loop until all idle callbacks have been called. This will update the display of windows but not process events caused by the user.

**validate**() → bool

validate the data

This method is called automatically to validate the data before the dialog is destroyed. By default, it always validates OK.

**wait_variable**(*name='PY_VAR'*)

Wait until the variable is modified.

A parameter of type IntVar, StringVar, DoubleVar or BooleanVar must be given.

**wait_visibility**(*window=None*)

Wait until the visibility of a WIDGET changes (e.g. it appears).

If no parameter is given self is used.

**wait_window**(*window=None*)

Wait until a WIDGET is destroyed.

If no parameter is given self is used.

**waitvar**(*name='PY_VAR'*)

Wait until the variable is modified.

A parameter of type IntVar, StringVar, DoubleVar or BooleanVar must be given.

**winfo_atom**(*name*, *displayof=0*)

Return integer which represents atom NAME.

**winfo_atomname**(*id*, *displayof=0*)

Return name of atom with identifier ID.

**winfo_cells**()

Return number of cells in the colormap for this widget.

**winfo_children**()

Return a list of all widgets which are children of this widget.

**winfo_class**()

Return window class name of this widget.

**winfo_colormapfull**()

Return True if at the last color request the colormap was full.

**winfo_containing**(*rootX*, *rootY*, *displayof=0*)

Return the widget which is at the root coordinates ROOTX, ROOTY.

**winfo_depth**()

Return the number of bits per pixel.

**winfo_exists**()

Return true if this widget exists.

**winfo_fpixels**(*number*)

Return the number of pixels for the given distance NUMBER (e.g. “3c”) as float.

**winfo_geometry**()

Return geometry string for this widget in the form “widthxheight+X+Y”.

**winfo_height**()

Return height of this widget.

**winfo_id**()

Return identifier ID for this widget.

**winfo_interps**(*displayof=0*)

Return the name of all Tcl interpreters for this display.

**winfo_ismapped**()

Return true if this widget is mapped.

**winfo_manager**()

Return the window manager name for this widget.

**winfo_name**()

Return the name of this widget.

**winfo_parent**()

Return the name of the parent of this widget.

**winfo_pathname**(*id*, *displayof=0*)

Return the pathname of the widget given by ID.

**winfo_pixels**(*number*)

Rounded integer value of winfo_fpixels.

**winfo_pointerx**()

Return the x coordinate of the pointer on the root window.

**winfo_pointerxy**()

Return a tuple of x and y coordinates of the pointer on the root window.

**winfo_pointery**()

Return the y coordinate of the pointer on the root window.

**winfo_reqheight**()

Return requested height of this widget.

**winfo_reqwidth**()

Return requested width of this widget.

**winfo_rgb**(*color*)

Return a tuple of integer RGB values in range(65536) for color in this widget.

**winfo_rootx**()

Return x coordinate of upper left corner of this widget on the root window.

**winfo_rooty**()

Return y coordinate of upper left corner of this widget on the root window.

**winfo_screen**()

Return the screen name of this widget.

**winfo_screencells**()

Return the number of the cells in the colormap of the screen of this widget.

**winfo_screendepth**()

Return the number of bits per pixel of the root window of the screen of this widget.

**winfo_screenheight**()

Return the number of pixels of the height of the screen of this widget in pixel.

**winfo_screenmmheight**()

Return the number of pixels of the height of the screen of this widget in mm.

**winfo_screenmmwidth**()

Return the number of pixels of the width of the screen of this widget in mm.

**winfo_screenvisual**()

Return one of the strings directcolor, grayscale, pseudocolor, staticcolor, staticgray, or truecolor for the default colormodel of this screen.

**winfo_screenwidth**()

Return the number of pixels of the width of the screen of this widget in pixel.

**winfo_server**()

Return information of the X-Server of the screen of this widget in the form "XmajorRminor vendor vendorVersion".

**winfo_toplevel**()

Return the toplevel widget of this widget.

**winfo_viewable**()

Return true if the widget and all its higher ancestors are mapped.

**winfo_visual**()

Return one of the strings directcolor, grayscale, pseudocolor, staticcolor, staticgray, or truecolor for the colormodel of this widget.

**winfo_visualid**()

Return the X identifier for the visual for this widget.

**winfo_visualsavailable**(*includeids=False*)

Return a list of all visuals available for the screen of this widget.

Each item in the list consists of a visual name (see winfo_visual), a depth and if includeids is true is given also the X identifier.

**winfo_vrootheight**()

Return the height of the virtual root window associated with this widget in pixels. If there is no virtual root window return the height of the screen.

**winfo_vrootwidth**()

Return the width of the virtual root window associated with this widget in pixel. If there is no virtual root window return the width of the screen.

**winfo_vrootx**()

Return the x offset of the virtual root relative to the root window of the screen of this widget.

**winfo_vrooty**()

Return the y offset of the virtual root relative to the root window of the screen of this widget.

**winfo_width**()

Return the width of this widget.

**winfo_x**()

Return the x coordinate of the upper left corner of this widget in the parent.

**winfo_y**()

Return the y coordinate of the upper left corner of this widget in the parent.

**withdraw**()

Withdraw this widget from the screen such that it is unmapped and forgotten by the window manager. Re-draw it with wm_deiconify.

**wm_aspect**(*minNumer=None*, *minDenom=None*, *maxNumer=None*, *maxDenom=None*)

Instruct the window manager to set the aspect ratio (width/height) of this widget to be between MINNUMER/MINDENOM and MAXNUMER/MAXDENOM. Return a tuple of the actual values if no argument is given.

**wm_attributes**(**args*)

This subcommand returns or sets platform specific attributes

The first form returns a list of the platform specific flags and their values. The second form returns the value for the specific option. The third form sets one or more of the values. The values are as follows:

On Windows, -disabled gets or sets whether the window is in a disabled state. -toolwindow gets or sets the style of the window to toolwindow (as defined in the MSDN). -topmost gets or sets whether this is a topmost window (displays above all other windows).

On Macintosh, XXXXX

On Unix, there are currently no special attribute values.

**wm_client**(*name=None*)

Store NAME in WM_CLIENT_MACHINE property of this widget. Return current value.

**wm_colormapwindows**(**wlist*)

Store list of window names (WLIST) into WM_COLORMAPWINDOWS property of this widget. This list contains windows whose colormaps differ from their parents. Return current list of widgets if WLIST is empty.

**wm_command**(*value=None*)

Store VALUE in WM_COMMAND property. It is the command which shall be used to invoke the application. Return current command if VALUE is None.

**wm_deiconify**()

Deiconify this widget. If it was never mapped it will not be mapped. On Windows it will raise this widget and give it the focus.

**wm_focusmodel**(*model=None*)

Set focus model to MODEL. “active” means that this widget will claim the focus itself, “passive” means that the window manager shall give the focus. Return current focus model if MODEL is None.

**wm_forget**(*window*)

The window will be unmapped from the screen and will no longer be managed by wm. toplevel windows will be treated like frame windows once they are no longer managed by wm, however, the menu option configuration will be remembered and the menus will return once the widget is managed again.

**wm_frame**()

Return identifier for decorative frame of this widget if present.

**wm_geometry**(*newGeometry=None*)

Set geometry to NEWGEOMETRY of the form =widthxheight+x+y. Return current value if None is given.

**wm_grid**(*baseWidth=None*, *baseHeight=None*, *widthInc=None*, *heightInc=None*)

Instruct the window manager that this widget shall only be resized on grid boundaries. WIDTHINC and HEIGHTINC are the width and height of a grid unit in pixels. BASEWIDTH and BASEHEIGHT are the number of grid units requested in Tk_GeometryRequest.

**wm_group**(*pathName=None*)

Set the group leader widgets for related widgets to PATHNAME. Return the group leader of this widget if None is given.

**wm_iconbitmap**(*bitmap=None*, *default=None*)

Set bitmap for the iconified widget to BITMAP. Return the bitmap if None is given.

Under Windows, the DEFAULT parameter can be used to set the icon for the widget and any descendants that don't have an icon set explicitly. DEFAULT can be the relative path to a .ico file (example: root.iconbitmap(default='myicon.ico') ). See Tk documentation for more information.

**wm_iconify**()

Display widget as icon.

**wm_iconmask**(*bitmap=None*)

Set mask for the icon bitmap of this widget. Return the mask if None is given.

**wm_iconname**(*newName=None*)

Set the name of the icon for this widget. Return the name if None is given.

**wm_iconphoto**(*default=False*, **args*)

Sets the titlebar icon for this window based on the named photo images passed through args. If default is True, this is applied to all future created toplevels as well.

The data in the images is taken as a snapshot at the time of invocation. If the images are later changed, this is not reflected to the titlebar icons. Multiple images are accepted to allow different images sizes to be provided. The window manager may scale provided icons to an appropriate size.

On Windows, the images are packed into a Windows icon structure. This will override an icon specified to wm_iconbitmap, and vice versa.

On X, the images are arranged into the _NET_WM_ICON X property, which most modern window managers support. An icon specified by wm_iconbitmap may exist simultaneously.

On Macintosh, this currently does nothing.

**wm_iconposition**(*x=None*, *y=None*)

Set the position of the icon of this widget to X and Y. Return a tuple of the current values of X and X if None is given.

**wm_iconwindow**(*pathName=None*)

Set widget PATHNAME to be displayed instead of icon. Return the current value if None is given.

**wm_manage**(*widget*)

The widget specified will become a stand alone top-level window. The window will be decorated with the window managers title bar, etc.

**wm_maxsize**(*width=None*, *height=None*)

Set max WIDTH and HEIGHT for this widget. If the window is gridded the values are given in grid units. Return the current values if None is given.

**wm_minsize**(*width=None*, *height=None*)

Set min WIDTH and HEIGHT for this widget. If the window is gridded the values are given in grid units. Return the current values if None is given.

**wm_overrideredirect**(*boolean=None*)

Instruct the window manager to ignore this widget if BOOLEAN is given with 1. Return the current value if None is given.

**wm_positionfrom**(*who=None*)

Instruct the window manager that the position of this widget shall be defined by the user if WHO is "user", and by its own policy if WHO is "program".

**wm_protocol**(*name=None*, *func=None*)

Bind function FUNC to command NAME for this widget. Return the function bound to NAME if None is given. NAME could be e.g. "WM_SAVE_YOURSELF" or "WM_DELETE_WINDOW".

**wm_resizable**(*width=None*, *height=None*)

Instruct the window manager whether this width can be resized in WIDTH or HEIGHT. Both values are boolean values.

**wm_sizefrom**(*who=None*)

Instruct the window manager that the size of this widget shall be defined by the user if WHO is "user", and by its own policy if WHO is "program".

**wm_state**(*newstate=None*)

Query or set the state of this widget as one of normal, icon, iconic (see wm_iconwindow), withdrawn, or zoomed (Windows only).

**wm_title**(*string=None*)

Set the title of this widget.

**wm_transient**(*master=None*)

Instruct the window manager that this widget is transient with regard to widget MASTER.

**wm_withdraw**()

Withdraw this widget from the screen such that it is unmapped and forgotten by the window manager. Re-draw it with wm_deiconify.

### lost_and_found.ui.widgets.dialog.DialogViewModel

**class** lost_and_found.ui.widgets.dialog.**DialogViewModel**(*title: str*, *body:* ViewModel)

Bases: *ViewModel*

Base class for dialog view-models supporting validation/apply.

**__init__**(*title: str*, *body:* ViewModel) → None

#### Methods

| | |
|---|---|
| *__init__*(title, body) | |
| apply() | |
| draw(parent) | |
| validate() | |

### lost_and_found.ui.widgets.entry

Text entry view and view-model binding to a *Property*.

#### Classes

| | |
|---|---|
| *EntryView*(vm) | Render a tkinter *Entry* bound bidirectionally to *text*. |
| *EntryViewModel*(text) | View-model that wraps a *Property[str]* for an entry widget. |

### lost_and_found.ui.widgets.entry.EntryView

**class** lost_and_found.ui.widgets.entry.**EntryView**(*vm: T*)

Bases: *View*[*EntryViewModel*]

Render a tkinter *Entry* bound bidirectionally to *text*.

**__init__**(*vm: T*) → None

#### Methods

| | |
|---|---|
| *__init__*(vm) | |
| draw(parent) | |

### lost_and_found.ui.widgets.entry.EntryViewModel

**class** lost_and_found.ui.widgets.entry.**EntryViewModel**(*text:* Property*[str]*)

Bases: *ViewModel*

View-model that wraps a *Property[str]* for an entry widget.

**__init__**(*text:* Property*[str]*)

#### Methods

| | |
|---|---|
| *__init__*(text) | |
| draw(parent) | |

### lost_and_found.ui.widgets.grid

Grid layout helpers for composing views in rows/columns.

### Classes

| | |
|---|---|
| *GridView*(vm) | Render child view-models into a grid using tkinter's *grid*. |
| *GridViewModel*(*rows) | View-model holding rows of child view-models for a grid layout. |

### lost_and_found.ui.widgets.grid.GridView

**class** lost_and_found.ui.widgets.grid.**GridView**(*vm: T*)

Bases: *View*[*GridViewModel*]

Render child view-models into a grid using tkinter's *grid*.

**__init__**(*vm: T*) → None

#### Methods

| | |
|---|---|
| *__init__*(vm) | |
| draw(parent) | |

### lost_and_found.ui.widgets.grid.GridViewModel

**class** lost_and_found.ui.widgets.grid.**GridViewModel**(**rows: tuple[*ViewModel, ...]*)

Bases: *ViewModel*

View-model holding rows of child view-models for a grid layout.

**__init__**(**rows: tuple[*ViewModel, ...]*) → None

#### Methods

| | |
|---|---|
| *__init__*(*rows) | |
| draw(parent) | |

### lost_and_found.ui.widgets.hlist

Horizontal list helpers for arranging children left-to-right.

### Classes

| | |
|---|---|
| *HorizontalListView*(vm) | Render child view-models using tkinter's *pack* side="left". |
| *HorizontalListViewModel*(*children) | View-model holding child view-models arranged horizontally. |

### lost_and_found.ui.widgets.hlist.HorizontalListView

**class** lost_and_found.ui.widgets.hlist.**HorizontalListView**(*vm: T*)

Bases: *View*[*HorizontalListViewModel*]

Render child view-models using tkinter's *pack* side="left".

**__init__**(*vm: T*) → None

#### Methods

| | |
|---|---|
| *__init__*(vm) | |
| draw(parent) | |

### lost_and_found.ui.widgets.hlist.HorizontalListViewModel

**class** lost_and_found.ui.widgets.hlist.**HorizontalListViewModel**(**children:* ViewModel)

Bases: *ViewModel*

View-model holding child view-models arranged horizontally.

**__init__**(**children:* ViewModel)

#### Methods

| | |
|---|---|
| *__init__*(*children) | |
| draw(parent) | |

### lost_and_found.ui.widgets.label

Simple label view and view-model.

#### Classes

| | |
|---|---|
| *LabelView*(vm) | Render a tkinter *Label* with the provided text. |
| *LabelViewModel*(text) | View-model for a static text label. |

### lost_and_found.ui.widgets.label.LabelView

**class** lost_and_found.ui.widgets.label.**LabelView**(*vm: T*)

Bases: *View*[*LabelViewModel*]

Render a tkinter *Label* with the provided text.

**__init__**(*vm: T*) → None

#### Methods

| | |
|---|---|
| *__init__*(vm) | |
| draw(parent) | |

### lost_and_found.ui.widgets.label.LabelViewModel

**class** lost_and_found.ui.widgets.label.**LabelViewModel**(*text: str*)

Bases: *ViewModel*

View-model for a static text label.

**__init__**(*text: str*)

#### Methods

| |
|---|
| *__init__*(text) |
| draw(parent) |

### lost_and_found.ui.widgets.option_menu

Option menu (dropdown) binding to a *Property* with formatting.

#### Classes

| | |
|---|---|
| *OptionMenuView*(vm) | Render a tkinter *OptionMenu* and keep *selected* in sync. |
| *OptionMenuViewModel*(selected, formatter, ...) | View-model representing a selected option and available choices. |

### lost_and_found.ui.widgets.option_menu.OptionMenuView

**class** lost_and_found.ui.widgets.option_menu.**OptionMenuView**(*vm: T*)

Bases: *View*[*OptionMenuViewModel*], Generic

Render a tkinter *OptionMenu* and keep *selected* in sync.

**__init__**(*vm: T*) → None

#### Methods

| |
|---|
| *__init__*(vm) |
| draw(parent) |

### lost_and_found.ui.widgets.option_menu.OptionMenuViewModel

**class** lost_and_found.ui.widgets.option_menu.**OptionMenuViewModel**(*selected:* Property, *formatter: Callable[[T], str]*, **options: T*)

Bases: *ViewModel*, Generic

View-model representing a selected option and available choices.

**__init__**(*selected:* Property, *formatter: Callable[[T], str]*, **options: T*)

#### Methods

| |
|---|
| *__init__*(selected, formatter, *options) |
| draw(parent) |

### lost_and_found.ui.widgets.table

Table view backed by a *ValueObservable* of rows.

#### Classes

| | |
|---|---|
| *TableView*(vm) | Render a *ttk.Treeview* and keep selection in sync with the view-model. |
| *TableViewModel*(headers, rows, formatter) | View-model exposing table headers, rows and selected items. |

### lost_and_found.ui.widgets.table.TableView

**class** lost_and_found.ui.widgets.table.**TableView**(*vm: T*)

Bases: *View*[*TableViewModel*], Generic

Render a *ttk.Treeview* and keep selection in sync with the view-model.

**__init__**(*vm: T*) → None

#### Methods

| |
|---|
| *__init__*(vm) |
| draw(parent) |

### lost_and_found.ui.widgets.table.TableViewModel

**class** lost_and_found.ui.widgets.table.**TableViewModel**(*headers: tuple[str, ...]*, *rows: ValueObservable[ImmutableList]*, *formatter: Callable[[T], tuple[str, ...]]*)

Bases: *ViewModel*, Generic

View-model exposing table headers, rows and selected items.

**__init__**(*headers: tuple[str, ...]*, *rows: ValueObservable[ImmutableList]*, *formatter: Callable[[T], tuple[str, ...]]*) → None

#### Methods

| |
|---|
| *__init__*(headers, rows, formatter) |
| draw(parent) |

### lost_and_found.ui.widgets.vlist

Vertical list helpers for stacking child view-models top-to-bottom.

#### Classes

| | |
|---|---|
| *VerticalListView*(vm) | Render child view-models using tkinter's *pack* side="top". |
| *VerticalListViewModel*(*children) | View-model holding children arranged vertically. |

### lost_and_found.ui.widgets.vlist.VerticalListView

**class** lost_and_found.ui.widgets.vlist.**VerticalListView**(*vm: T*)

Bases: *View*[*VerticalListViewModel*]

Render child view-models using tkinter's *pack* side="top".

**__init__**(*vm: T*) → None

#### Methods

| |
|---|
| *__init__*(vm) |
| draw(parent) |

### lost_and_found.ui.widgets.vlist.VerticalListViewModel

**class** lost_and_found.ui.widgets.vlist.**VerticalListViewModel**(**children:* ViewModel)

Bases: *ViewModel*

View-model holding children arranged vertically.

**__init__**(**children:* ViewModel)

#### Methods

| |
|---|
| *__init__*(*children) |
| draw(parent) |

# PYTHON MODULE INDEX

## l

# INDEX

## Symbols

## A



## B



## C

## D



## E



## F



## G

## H



## I



## K



## L

## M



## N



## O

## P



## Q



## R



## S

## T



## U

## V



## W